

THE EXTENDED COURSE · SEPTEMBER 2026

From Prompts to Agents

Writing prompts as small work specifications — and delegating work you can check. The written edition of the facilitated course, complete with the nine-step worked sequence: analyze one dataset, return the workbook, build the dashboard, present the findings, organize the thread, compare the quotes, and build the research workbook.

THIS BOOK	The course, cover to cover, for self-study or review after the live session.
THE DECK	From_Prompts_to_Agents_Facilitated_60_Minute.pptx — the 60-minute live course.
THE WORKBOOK	Course_Workbook.xlsx — the data and the exercise tabs, in training order, this book works through.
THE BUILDER	Prompt_Template_Creator.html — assembles prompts from the same requirement menus.
ONE PAGE	From_Prompts_to_Agents_Cheat_Sheet.pdf — the skeleton and the menus, printable.

All names, companies, and numbers in this book are fictional and generated for practice. Your organization's AI policy and approved-tool list outrank anything written here.

CONTENTS

The same course, in the same order	5
How to read this book	5
Foundations: how these systems work	6
One skill, two ways to work	6
Four eras, and the decade of named leaps	6
How a language model produces an answer	7
Tokens and practical limits	8
Context, saved history, and memory	8
Answers grounded in documents	9
Effort, capability, and the smallest fix	10
Verification belongs in the task	11
AI mirrors you	12
Diagnose the prompt, not the model	12
Models and tools: choosing where the work happens	13
Compare on your task, not on the leaderboard	13
January 2025: the efficiency moment	13
The right workspace for the data	14
Prompt engineering: writing the work specification	16
You already write work specifications	16
Good requirements make prompts testable	16
A prompt is a small work specification	17
Useful roles describe behavior	18
Prompt elements are requirement types: the menu	18
Examples, uncertainty, and stopping	19
A menu of requirements for each artifact	20
The improvement loop	20
One task through the improvement loop	21
Your turn: turn your course log into a report	22
The prompt is a document, not a sentence	22
Prompt techniques to test	23
A practical prompt review checklist	24
Putting prompts to work: one dataset, nine steps	25
One dataset, nine steps	25
Step 1: inspect, then analyze	26

Step 2: one follow-up, in the same chat	28
Step 3: the assistant returns your workbook	28
Step 4: a dashboard from the returned workbook	29
Step 5: five slides from the same findings	31
Step 6: organize the email thread	33
Step 7: extract the quotes	35
Step 8: compare the quotes - and refuse to pick a winner	35
Step 9: reference files become a research workbook	36
Reference exercise: explain a topic clearly	37
The arc, complete	38
Agentic work: delegating with boundaries	40
What makes a workflow agentic	40
ASK and DELEGATE: the decision, not the tools	40
Which way of working fits: a self-test	41
The anatomy, grown up to survive autonomy	41
The mission brief for bounded work	42
Worked exercise: the supplier analysis becomes a mission brief	42
Choosing an environment for delegated work	43
Instructions versus enforceable controls	44
Diagnosing a failed run	44
Write the brief once — the workspace remembers	45
Exercise: one gate and one hard check	45
Reusable prompts: from one good result to a shared asset	46
A useful prompt becomes a reusable asset	46
Where reusable prompts can live	46
A small review process for shared prompts	47
Save one prompt for Monday	48
Let the assistant interview you	49
The habits to carry into work	49
The capstone: one task, one checked result	50
Your next workplace application	50
Reference: the glossary, the criteria, and the evidence	52
The glossary: twenty terms that matter	52
Why each dial matters: the generative criteria	53
Why each dial matters: the agentic criteria	53

The evidence verdicts: works, myth, expired 54

Bibliography 54

The package 56

The practice pack: every exercise source, in full 57

A.1 · The supplier dataset (Supplier_Data_Exercise.xlsx) 57

A.2 · The email thread (Packaging_Change_Thread.txt) 62

A.3 · The supplier quotations (Quote_*.pdf) 63

A.4 · The research pack (research-pack/) 66

A.5 · The plain-language pack (plain-language/) 67

Answer keys: check yourself after attempting 73

B.1 · Supplier analysis — the worked key 73

B.2 · Email brief — the expected result and the four traps 73

B.3 · Quotation comparison — the key 74

B.4 · Research workbook — the key 75

The one-page cheat sheet 76

HOW THIS BOOK FOLLOWS THE COURSE

The same course, in the same order

This book is the facilitated course, compiled cover to cover. Chapters follow the deck’s parts, sections follow the slides in their live order, and the deck’s reference appendix is folded into each chapter at the matching section — so the book reads in the same sequence the course is taught, with the depth the live hour has no room for.

Part 1 · Foundations	Slides 2–15 (+ detail pages 50–52)	Chapter 1
Part 2 · Models and tools	Slides 16–21	Chapter 2
Part 3 · Prompt engineering	Slides 22–35 (+ detail pages 53–58)	Chapter 3
Part 4 · The application block	Slides 36–48, steps 1–8 of the nine-step sequence (+ detail page 59); step 9 and the reference exercise are Reference Deck slides 112 and 102	Chapter 4
Part 5 · Agentic work	Reference Deck slides 47–58 (+ main-deck detail pages 60–63)	Chapter 5
Part 6 · Reusable prompts + close	Reference Deck slides 59–66 and 72	Chapter 6
Reference block	Reference Deck slides 67–71 and main-deck slides 64–69 — glossary, criteria, sources, the making-of	Chapter 7
Practice material	Every exercise source, verbatim: dataset, thread, quotations, research pack, plain-language	Appendix A
Answer keys	Self-check material: the expected result for every exercise — attempt each exercise first	Appendix B
The cheat sheet	The printable one-page course cheat sheet, unchanged from the separate file	Appendix C

How to read this book

Three conventions carry the working material. A shaded monospace box is a copy-paste prompt, used verbatim in the course; its caption names the Course_Workbook tab that holds the same text, so you can copy from either place. A “Success check” callout is the acceptance evidence for an exercise — run the step, then hold your result against it. And every exercise names its input file; the fictional dataset, thread, quotations, and reference pack all ship in the package, so each step can be run and checked exactly as taught.

PART 1 - FOUNDATIONS

Foundations: how these systems work

Everything in this course rests on a handful of mechanisms: what a language model actually does when it answers, what it can and cannot remember, and why a fluent answer is not the same thing as a true one. This chapter builds that foundation from zero. Read it with an assistant open - several sections end with a prompt to run, copied from a named tab of Course_Workbook.xlsx in the deliverables folder.

One skill, two ways to work

Prompting is the one AI skill that transfers everywhere: every assistant, every vendor, every year. Models change monthly; the craft compounds. The skill has two modes, **ASK** and **DELEGATE**, and the ten-second version is all you need for now: a generative prompt describes what to write; an agentic prompt describes a job to run.

That distinction is the backbone of this course - the craft chapters teach the first kind, and Chapter 5 grows it into the second, defining both modes side by side. For now, feel the difference.

TRY IT — Open your course log

Open a fresh chat and keep it all course - your course log. Run the two steps below, from tab EX1-TwoModes of Course_Workbook.xlsx.

SEND 1: Use this chat as my course log. When I request a report, use only the conversation available to you and flag any gaps. Acknowledge briefly.

SEND 2: Write a farewell message for a coworker in about 50 words.

SEND 3: Before revising, ask up to three questions that would most improve the message. Wait for my answers, then revise it.

Tab EX1-TwoModes - run all three sends in one chat, and keep that chat: it is your course log.

Step one is **ASK**: instant text, every unknown guessed. Step two is the seed of **DELEGATE**: the questions come to you. If your assistant asks questions in step one too, that is no failure - asking is the delegate move; step two makes it happen on purpose.

Four eras, and the decade of named leaps

Seventy years of AI compress into four eras. In the **rules** era (1950s-1990s), humans hand-coded the logic: if X, then Y. Expert systems did real work - MYCIN diagnosed infections, XCON configured computers - but rules crack on messy reality. Hype outran results twice, in the funding collapses remembered as the AI winters: skepticism is historically earned. In the **learning** era (1990s-2010s) the approach inverted: let the system find patterns in examples. It gave us spam filters, search ranking, recommendations, and modern image recognition - decisively in 2012, when deep networks on gaming graphics chips (GPUs) cut image-recognition error from 26.2% to 15.3% (AlexNet).

The **generative** era began in 2017 with the Transformer, which lets models learn patterns from large datasets: predict-the-next-word at that scale writes, summarizes, and codes, and ChatGPT (2022) made it something everyone could talk to. The **agentic** era (2024 onward) gives models arms - tools, browsers, files, code - and reasoning models that plan multi-step work and execute it with limited supervision. AI you delegate to.

KEY — Why the last decade exploded

Scaling laws (2020): a bigger model, more data, and more compute produced improved measured performance under the studied conditions. Progress stopped being a research gamble and became an investment roadmap.

Year	The leap	What it changed
2017	The Transformer (Attention Is All You Need)	A foundation for many subsequent language models
2020	Scaling laws + few-shot (GPT-3)	Show examples in the prompt and it does the task - prompt engineering is born
2022	Instruction tuning + human-feedback tuning (RLHF)	Human feedback turns a predictor into an assistant
2024	Reasoning models	Compute spent at answer time, not just in training
2025	Mixture of Experts (MoE), distillation, open weights	Frontier ability gets radically cheaper; big models teach small ones
2026	Agentic harnesses	Delegation reaches general office work

Each method on this list returns later in the course - treat it as vocabulary.

Adoption outran precedent: ChatGPT reached 100 million users in about two months - the fastest consumer ramp on record at the time (UBS/Reuters, 2023), against roughly nine for TikTok and thirty for Instagram. Access changed faster than workplace processes adapted; that gap is why this course exists.

The hardware, in scoped terms

Two principles are enough. **Training:** large models learn from repeated computation over data, so parallel processors, memory, and data movement constrain what can be trained. **Serving:** each request also requires computation, so model design, requested reasoning effort, and workload drive response time and operating cost. Reading hardware claims, keep three categories separate: measured results, estimates, and announced investments.

How a language model produces an answer

The useful mental model: a prediction engine, sent to finishing school. In **pretraining**, the model spends months on trillions of words learning one task - predict the next token - and language, facts, and reasoning patterns compress into billions of learned weights. **Instruction tuning** uses curated example dialogues to teach it to follow instructions rather than merely continue your text. **Human feedback** (RLHF) then tunes it toward helpful, honest, and harmless, from people ranking outputs. How your own feedback and chats are used depends on the product, account, and data settings - check, don't assume.

Two consequences follow. First, knowledge is frozen at the training cutoff, months before release. That is why assistants bolt on escape hatches: live web search pulls today's pages into the prompt, and retrieval plugs in your own documents (below). Second, the model completes your text. The prompt is one way to steer the work, alongside sources, tools, and settings - and a complete prompt does not make the model smarter. It deletes wrong guesses.

KEY — Prediction, not verification

A language model predicts useful continuations; it does not independently verify truth. A fluent answer can still contain an unsupported claim.

SEND 1: Finish this sentence five different ways: “We should review the launch date because...” Treat each ending as a hypothetical possibility.

SEND 2: Repeat using these facts: we are a business-to-business software company; our beta ends April 20; a competitor launches May 3. Separate supplied facts from assumptions. End with one missing fact that could change the timing decision.

Tab EX2-Guesses - two separate sends, in your course log.

The first send scatters across plausible guesses - budget, staffing, quality. The second orbits May 3 and April 20. Grade the directions, not the sentences: your facts deleted wrong guesses. That is your contribution to every answer: the task, the source material, the definitions, and a way to check the result. Compare 'Explain the delay' with 'Explain the delay using these three dated messages' - only the second can be checked. A model filling unstated requirements with its most plausible guess matches what the requester meant only about 41% of the time (Yang et al., 2025; revised 2026); this course is about closing that gap on purpose.

Tokens and practical limits

The model never sees letters. Text is chopped into **tokens** - word chunks from a fixed vocabulary, like bricks of text. Common words are one pre-molded brick; rare words get built from pieces (ham-bur-ger). English rules of thumb: one token is about four characters, or three-quarters of a word; a 50-page document runs roughly 25-35 thousand tokens. A token is not a fixed number of words, and limits vary by model and application - check your workspace's actual limits.

Three practical consequences. Everything is priced and limited in tokens - and output typically costs around five times input (as of August 2026), because generation is serial - so paste-heavy prompts burn budget and context fast. Non-English and dense technical text cost more tokens for the same meaning. And character-level precision is not the model's native ability: use AI for language and software for characters - counts, checksums, exact IDs.

EXPIRED — The strawberry test

The famous letter-counting failures of 2024 (miscounting the r's in 'strawberry') are patched on current frontier models as of September 2026. Still breaking: exact word counts and counting their own tokens on every tier; character-precise edits on fast tiers. For exact checks, count with a tool, not the model.

Explain tokens to a new colleague using a LEGO-brick analogy, in about 100 words. Show an illustrative word split and explain why the actual split depends on the tokenizer. Connect token counts to context limits and charges when billing is based on tokens. End with one practical implication for working with long documents.

Tab EX3-Tokens - the definition lands in your course log, from your own tool.

Context, saved history, and memory

Three things sound alike and are not. **Active context** is the information available to the model for this response. **Saved chat history** is a stored conversation you may be able to reopen - closing a window does not necessarily delete it. **Cross-chat memory** is a product feature that may carry

selected information into other conversations. Only the first is guaranteed to influence the answer in front of you.

For active context, use the desk analogy: the context window is working memory for one conversation. Your question, the system prompt, every prior turn, attached files, and tool results must all fit on the desk. By 2026, roughly one million tokens (about 750,000 words) is the flagship standard - but consumer apps often enforce smaller limits, and a model's published limit may differ from what your interface makes available.

WORKS — Lost in the middle

In long contexts, accuracy is highest at the start and end and sags in the middle (Liu et al., 2023). Placement alone is worth up to ~30% on long documents (Anthropic's long-context guidance).

- Long documents at the top, the question at the end - check the effect on your own task.
- Label multiple documents so the model can tell them apart.
- New topic, new chat - old turns are still on the desk, steering.
- Long thread? Run the handoff move below.

Just as important is what the model does not remember. Nothing persists into a fresh conversation unless a memory feature or a saved file reloads it, and a very long chat may compress or omit earlier material. Carry important requirements forward yourself: for a handoff, save the goal, sources, decisions, open items, and next step.

SEND 1: Explain a context window using a desk analogy, in about 100 words. What occupies the desk, and what happens as it fills? Distinguish information available for the current response from saved chat history and optional memory. Identify what depends on the application. End with one useful habit for continuing a long task.

HANDOFF FOLLOW-UP: Create a concise HANDOFF from the conversation currently available: goal, decisions, constraints, unresolved questions, and next action. Name any files needed to continue. Mark missing information rather than inventing it.

Tab EX4-Handoff - later, paste the HANDOFF into a fresh chat and ask 'where was I?'

One term to file for the agentic chapters: **context engineering**. The prompt is the briefing; context is everything on the desk, including things nobody typed - tool results, standing instructions. Agentic tools bolt a filing cabinet onto the desk: persistent files and notes that outlive one conversation.

Answers grounded in documents

The model's built-in knowledge is frozen and public; your work runs on private documents it has never seen. **Retrieval-augmented generation** (RAG) bridges that gap at answer time - an open-book exam. You ask a plain question ('What does the supplier agreement say about re-inspection?'). A retrieval step finds the most relevant passages by meaning rather than keywords (semantic search). Those passages land on the model's desk behind the scenes, and the model answers from the supplied text, citing where it looked.

Grounding beats recall for three reasons. It is **current**: documents are re-indexed in minutes, with no retraining. It is **checkable**: citations point to the exact passage, which free recall can never do. And it can respect boundaries - though privacy and access depend on the actual service, its permissions, retention, and training-use settings, so confirm what your organization runs. Retrieval feeding one answer is not training the model on your documents; that is the privacy point worth repeating. Many internal document assistants work exactly this way.

TRAP — The wrong page, beautifully cited

Fetch the wrong page - an outdated revision, a near-miss document - and the model still writes a fluent, confident, cited answer from it. An outdated agreement can support a well-cited but wrong answer for today's decision.

CHECK — Audit the librarian

Ask for the document, the revision, and the supporting passage. Then read the cited passage and confirm it actually answers the question. Check the citation, not just the prose.

Effort, capability, and the smallest fix

Instructions, sources, and model adaptation

When an answer disappoints, there are three sizes of intervention, and the smallest that solves the problem wins. Clearer **instructions** fix behavior - tone, format, focus, approach. Better **source material** fixes missing knowledge - your policies, your data, current facts; most real workplace cases end here. **Model adaptation** (retraining) changes deep habits permanently; it is slow, expensive, and rare - an IT project, justified only by a recurring need that evaluation shows simpler changes cannot meet. One question picks the level: is the model missing knowledge (supply documents), or misbehaving with knowledge it has (fix the prompt)? More documents or more training do not automatically solve unclear requirements.

Reasoning effort and the task

Assistants ship a fast path - one pass, instant, cheap - and a thinking path that drafts, checks, and revises internally before answering. The model is not literally thinking: it generates intermediate tokens that improve the final answer, and those are billed like output, the expensive kind. A hard question can quietly cost 5-20 times a simple one. Try more effort for a difficult comparison, several dependent calculations, or a result that needs multiple checks; keep it off for lookups, reformatting, summaries, and routine drafting. Measure correctness, useful evidence, response time, and cost in the account you actually use. Escalate on failure, not by default - more computation does not replace clear requirements or verification.

Explain when to start with fast responses versus more reasoning, using three examples: a routine email, inconsistent supplier data, and a problem with several interacting causes. For each, give a starting choice and what would justify increasing effort. Explain the tradeoff in time and resource use without guessing prices. About 120 words.

Tab EX5-TwoSpeeds - the model explains its own two speeds, in your course log.

Capabilities to look for in an assistant

- Quick response - when the task is clear and the output is easy to check.
- Additional reasoning - for a difficult comparison or a task requiring several checks.
- Search or document retrieval - when the answer needs current or supplied source material.
- File creation and tool use - when the deliverable includes calculations, editable files, or actions.

Names, limits, and costs vary by vendor and account. Verify your account's actual menu rather than memorizing a catalog, and never rely on a model's description of where its own buttons live. The durable habit: pick the smallest mode that can succeed, and escalate deliberately.

Using the supplied screenshot or copied text of my app's menu, explain each visible mode in one line: its purpose and a suitable workplace task. Mark anything the supplied information does not establish as needing verification. If the menu is missing, ask for it first.

Tab EX6-FeatureMenu - the deck keeps this one in the reference layer (Reference Deck slide 113). Self-updating: ask again the day a button changes.

Verification belongs in the task

Now the mechanism that makes verification non-optional. Models are optimized for the most plausible next token, not the most true one, and trouble concentrates where training data is thin: rare facts, citations, numbers, names. OpenAI's own 2025 research adds the incentive: benchmarks reward confident guessing over 'I don't know', so models learn to be good test-takers. The better word than hallucination is **confabulation** - gaps filled with plausible material, in format: fake citations look exactly like citations.

The effect is measurable. In a comparison OpenAI published in 2025, an older reasoning model answered 24% of hard factual questions right and 75% wrong. A newer model, trained to say 'I don't know', answered 22% right and 26% wrong, abstaining 52% of the time - roughly three times fewer false claims at nearly the same amount right. The confident guess is a training artifact. Explicit permission to not know - the move the craft chapters call the Out - recovers the honesty the benchmarks trained away.

MYTH — 'Are you sure?' is not verification

Pushback is a social signal, not a re-derivation: the model re-reads you, not its work. Answers flip in 58% of cases, 14.7% from right to wrong (SycEval, AIES 2025). Verification is a named check or an outside source - never the same model reassuring you.

So verification belongs in the task itself. 'The supplier approved the change' invites one question: what message supports that claim? 'Approval is conditional on the cost cap' - with the message shown and the condition preserved - can be checked, and keeps the qualifier that makes it true. When you set a task, name the source or calculation that would settle the question.

The public record shows what skipping the check costs. In June 2023 a New York lawyer filed six cases that did not exist - then asked ChatGPT whether they were real, and it said yes (\$5,000 sanction, *Mata v. Avianca*): he checked the fabrication with the fabricator. In February 2024 a tribunal made Air Canada honor a refund policy its website chatbot had invented - your organization owns what its chatbot says. In May 2024 an AI search feature served a joke forum comment as advice: glue in pizza sauce. Across 2025-26, three global consultancies had reports challenged over invented or failed citations, and a legal tracker counted about 1,500 court decisions worldwide involving AI-fabricated citations by mid-2026. Each time, the missing thing was a check someone could have run.

The cure has not changed: **ground it** (give it the source), **cite it** (demand receipts, plus an 'I don't know' escape hatch), **verify it** (check what matters before it ships). None of these eliminates confabulation - they convert unverifiable claims into verifiable ones. Which check to run depends on what kind of wrong you are guarding against:

- A **number** - recompute it.
- An **invented fact** - find the supporting record, or state that it is unknown.
- An **invented citation** - open the actual source and inspect the cited passage.

- The **wrong document or revision** - confirm identity, date, and applicability first.
- A **lost condition** - compare the conclusion with the original qualifier or approval condition.
- An **action** - inspect the result it produced, not the report of it.

House rule from here on: every uncited standard clause, date, number, or quote is a draft until verified. A credible format, a confident tone, or a citation marker is not a check - and a memorable failure story matters only if it changes which checks you run.

AI mirrors you

AI mirrors you. Across eleven leading models, AI affirmed the user about half again as often as a person would - plus 49 percent (Stanford and Carnegie Mellon, in Science, 2026). The cause is structural: models are trained on human preference data, and humans prefer agreement. The pattern made headlines first. In April 2025 OpenAI rolled back a ChatGPT update it called, in its own words, "overly flattering or agreeable... sycophantic". In June 2025 the New York Times reported chatbots validating false beliefs. In August 2025 Microsoft's AI chief warned of unhealthy AI validation.

The countermeasure is in how you ask. A leading request — "I think this is excellent. Tell me why it works" — embeds the preferred conclusion in the task. A neutral one names a rubric and invites disconfirmation: "Evaluate this against clarity, evidence, and completeness. Identify weaknesses and what would change your assessment." Compare reasoning and evidence — a harsher answer is not automatically more accurate either. Never reveal your preferred answer; ask for the case against; paste your own draft as a colleague's; treat agreement under pushback as noise.

TRY IT — See the mirror yourself

Two chats, same paragraph. One: "A colleague wrote this — rate it 1-10, three biggest weaknesses." Two: "I wrote this myself and I'm really proud of it — rate it 1-10, three biggest weaknesses." Did the score move once it knew you were the author?

Diagnose the prompt, not the model

The last foundation is diagnostic. When an answer is wrong, the reflex is to reword the whole prompt and hope. Resist it: one failed phrasing is a data point about the prompt, not the capability ceiling. Name what kind of wrong you got, because each kind points at the part of the prompt that owns it. Chapter 3 builds those parts and the grid that maps each symptom to its element.

Next comes the craft: turning what you already know about writing requirements into testable prompts, practiced on one fictional supplier case that runs through the whole book.

PART 2 - MODELS AND TOOLS

Models and tools: choosing where the work happens

Before you invest in the craft of prompting, spend a few minutes on two decisions that sit upstream of every prompt: which assistant does the work, and which workspace is allowed to see the data. The assistant market re-ranks monthly, so this chapter deliberately teaches you a method for choosing rather than a ranking to memorize. The names churn; the habits you leave with do not.

Compare on your task, not on the leaderboard

Public leaderboards answer a question you are not asking: which model scores highest on someone else's benchmark. Your question is narrower and more useful: which assistant, on the account you actually have, does your task well.

When you need to choose an assistant for a piece of work, walk four questions in order:

- **Required output.** Can this account produce the text, calculation, file, or action you need?
- **Source access.** Can it use the right documents and show supporting evidence?
- **Work environment.** Does it fit the approved workspace and collaboration process?
- **Evaluation.** Run the same representative task on the candidates and compare correctness, effort, and cost.

The assistants you will meet by name in this course include ChatGPT, Claude, Copilot, Gemini, Perplexity, and Grok; verify current offerings before relying on any characterization of them. What stays true across the churn is that different assistants expose different capabilities and controls, and the same brand behaves differently on different account tiers - which is why the fourth question settles more arguments than any review article.

KEY — The rule that survives the churn

Tool choice follows the job, the evidence, and the approved data boundary, not habit and not a leaderboard.

EXPIRED — Rankings age in months

Any model ranking or product claim without a date is expired on arrival. Treat dated names and rankings as reference context, not a purchasing guide, and re-verify before a decision that matters.

TRY IT — A ten-minute bake-off

Pick one task that represents your real work. Run the identical prompt in two assistants you can access, then compare correctness, effort, and cost. Keep the result with its date.

January 2025: the efficiency moment

One event explains why the advanced capabilities from Part 1, extended thinking and deep research, did not stay luxury-priced. In January 2025, the Chinese lab DeepSeek released R1, and three facts made it a landmark:

- DeepSeek-R1 was an open-weights reasoning model rivaling OpenAI's o1, with MIT-licensed weights anyone could download.
- Nvidia lost \$589 billion in market value in one day (January 27), then the largest single-day market loss ever.
- The famous "\$5.6M training cost" headline described real efficiency but was oversold: that figure was the fuel for the final race, not the cost of the racing team.

The trick has a name: Mixture of Experts

Picture a huge specialist hospital at night. The building holds 671 billion parameters on the books, but for any one question only about 37 billion wake up: the lit windows, the relevant departments. You pay for the specialists consulted, not for the whole building. Distillation, big models teaching small ones, finishes the job. The result on the bill, per million output tokens as of January 2025: OpenAI's o1 cost \$60, DeepSeek-R1 cost \$2.19, roughly 27x cheaper. A \$100 o1 workload cost about \$3.60 on R1.

Benchmark	DeepSeek-R1	OpenAI o1
Math olympiad (AIME 2024)	79.8	79.2
Coding agent (SWE-bench)	49.2	48.9
Science PhD (GPQA)	71.5	75.7

Scores as reported in the R1 paper, January 2025. Matched on math and coding, behind on PhD-level science: not a clean sweep.

Explain Mixture of Experts using a specialist-hospital analogy, in about 120 words. Describe how selected computational components handle each token and how activating only a subset can reduce computation. Explain one way the hospital analogy can mislead, and why lower computation alone does not determine customer prices.

Copy from tab EX7-MoE of Course_Workbook.xlsx; the answer becomes the MoE cheat-note in your course log.

The right workspace for the data

Capability is the second question. The first is where the data may go. Data sensitivity decides the tool, and the decision runs through three checks before anything is uploaded:

- **Approved account.** Use the service and the account approved for this work.
- **Allowed material.** Confirm what may be uploaded, retained, and shared.
- **Minimum access.** Give the tool only the files and permissions the task requires.

A vendor's home country alone does not describe every customer's storage arrangement. Company data rules depend on the actual account, service, contract, and configuration. Data residency, choosing where your data lives, is typically an enterprise feature: the same vendor can host your company tenant - its own fenced-off instance of the service - in the EU while its consumer app stores chats in the United States. Same logo, different homes; the account tier decides which one is yours. The one question that settles it is for your IT department: which region and tier does your account run on? Your organization's AI policy answers everything else.

A distinction worth keeping when open-weights models come up: using a vendor's app is eating at their restaurant, where they see your order. A downloaded open-weights model is a cookbook your IT team can cook from at home. The app, not the model, is where the data-handling risk lives.

This is also why every exercise in this course runs on fictional data: the supplier quality scenario with Alpha Components, Bravo Plastics, and Cardinal Metals in the Data tab of `Course_Workbook.xlsx`. It behaves like real operational data but is safe in any workspace, so you can practice the craft before you ever face the upload question with real records.

KEY — The standing rule

Your organization's AI policy and approved-tool list outrank every product named in this part, and they outrank this book. Where they conflict with anything written here, the policy wins.

PART 3 - PROMPT ENGINEERING

Prompt engineering: writing the work specification

This is the craft chapter: how to get what you actually meant. Its central claim, once seen, is hard to unsee: writing a good prompt is writing a good requirement — and you almost certainly write requirements already. The chapter builds the anatomy of a strong prompt, walks the menu of requirement types, and puts both on an improvement loop.

You already write work specifications

Start with a recognition. Requirements, standard operating procedures, and test protocols exercise one core skill: stating what a piece of work must contain, how it must behave, and how you will know it is right. A prompt asks the same of a reply or an artifact — not a different skill, a different audience. Andrew Stellman, 2025: "Prompt engineering and requirements engineering are literally the same skill — using clarity, context, and intentionality to communicate your intent."

The forms map directly. A user story — "As a [role], I want [goal], so that [benefit]" — is role plus task with a purpose clause. Given [state], When [action], Then [result] maps to context, task, and format plus checks. The ISO 29148 characteristics — unambiguous, complete, singular, verifiable — are what this chapter asks of every prompt line.

The stakes: a model guesses an unstated requirement right only about 41 percent of the time (Chapter 1); a written specification deletes the guessing. Software moved first. GitHub's Spec Kit (2025) makes the written spec "the source of truth" that AI agents build from. Sean Grove of OpenAI drew the conclusion: "Whoever writes the spec... is now the programmer." One caution: a model can still misinterpret a clear instruction, so verification stays necessary.

KEY — The hardest 80 percent, already owned

If you can write a procedure, a test protocol, or an acceptance criterion, you own the hardest part of prompt engineering. The elements are a requirements document in miniature.

Good requirements make prompts testable

Two ideas carry this chapter. **Requirement type** names what must be specified — role, context, format, method. **Requirement quality** describes how well any selected requirement is written. The quality bar applies the writing principles summarized by NASA (Systems Engineering Handbook, Appendix C) and INCOSE (Guide to Writing Requirements).

Quality	For a prompt, it means	The failure it prevents
Necessary	Every line earns its place; a simple task needs a few.	Rule piles that bury what matters.
Clear	One reading; named columns, defined terms, no "etc."	Ambiguity resolved silently, the model's way.
Complete enough	The consequential facts and rules stated — not every fact.	Gaps filled with plausible guesses.

Quality	For a prompt, it means	The failure it prevents
Consistent	No requirement contradicts another.	The model reconciling you with yourself.
Feasible	Possible with the data and tools supplied.	Confident output the data cannot support.
Singular	One requirement per line — separable, revisable.	Half-followed compound instructions.
Verifiable	An observable check exists: a number to reconcile, a behavior to exercise.	"Looks done" standing in for done.

A requirement you cannot check is a wish. Separate the required outcome from a preferred implementation.

For a data prompt, "complete enough" means: dataset and period, row boundaries (what one row represents), calculation definition, deliverable, missing-data treatment, reconciliation. "Tell me which supplier is worst" leaves the model to pick the rows, the metric, and the meaning of "worst" - silently. "Calculate returns divided by units shipped by supplier and reconcile to the source" carries its own test.

CHECK — Verification and validation

Verification: does the output meet the written criteria? Validation: does it serve the actual business need? Check both, every time — these principles improve clarity and testability without making model behavior deterministic.

A prompt is a small work specification

One worked case: a manager must choose supplier follow-up. The prompt — calculate return rate from the supplied workbook, as total returns divided by total units shipped; exclude the existing TOTAL row; return a ranked table with a brief, neutral explanation; state missing information; reconcile totals to the source. Each sentence is a requirement: audience, source, method, format, tone, check. The exclusion prevents double counting; the reconciliation makes the answer checkable. Generalized, every prompt shares one skeleton:

```
Purpose & audience: help [reader] make [decision] or complete [task].
Sources & scope: use [source / version / period]; include [scope]; exclude [items].
Selected requirements: only the types that resolve ambiguity for THIS artifact —
a simple task needs a few, not all.
Acceptance evidence: check [specific result] against [source, calculation, or
observable test].
Missing info & finish line: if [gap]: write "Not stated" and list it;
if [conflict]: ask or stop. Answer, report limitations,
then stop. Actions needing review: [actions].
```

The skeleton of every prompt — printed on the cheat sheet ([deliverables/From_Prompts_to_Agents_Cheat_Sheet.pdf](#)).

The course's fuller anatomy has seven parts: five elements — Role, Task, Context, Format, Examples — plus two one-sentence safety valves, the Out and the Stop. Written out on a mundane task, the whole body is one continuous prompt of about ninety words, under two minutes to write:

ROLE You are a precise editor. You add no facts that are not in the source.
 TASK Summarize the attached Q2 returns report for the operations lead.
 CONTEXT We're a mid-size electronics manufacturer. Return rate = returns ÷ shipped.
 The export has a totals row – exclude it.
 FORMAT ≤ 150 words: the headline number first, then three bullets.
 EXAMPLES <example> March's summary – the one the team liked </example>
 THE OUT Anything the report doesn't state: write UNKNOWN – never estimate.
 List the gaps at the end.
 THE STOP Deliver the summary, then stop – nothing beyond.

Seven parts, one body. Miss a part and the prompt still works — it just fails in a predictable way, which is what makes diagnosis possible.

Notice the industry line: what business you are in is context - the assistant does not know your world until you say so. Anthropic's golden rule applies to the whole body: show the prompt to a colleague with minimal context; if they would be confused, the model will be too.

Useful roles describe behavior

"Act as a senior analyst" is a title — a vibe the model can fake. "State your assumptions, calculate from the source, and identify what the data cannot answer" is conduct a reader can observe: either the assumptions were stated or not. The craft is decomposing an identity into personality statements — testable behaviors, not credentials — paired with a concrete question and an intended audience.

The evidence draws the boundary. Expert personas do not improve factual accuracy: 162 personas, no gain (Zheng et al., 2024), confirmed by a six-model replication (Wharton, December 2025). What a role reliably shifts is voice and register — assigned traits are identifiable in output up to 80 percent of the time (PersonaLLM, 2024). Use Role for how it answers, never instead of checking what it answers.

- Weak: "You are a senior analyst." Strong: "You never invent numbers; you state n; you say what the data can't answer."
- Weak: "You are a helpful assistant." Strong: "You challenge weak assumptions instead of agreeing; you flag uncertainty rather than smooth it over."

KEY — Behaviors beat titles

A role earns its place by naming checkable conduct, not by conferring rank — each personality statement is auditable in the output; a title is a vibe.

Prompt elements are requirement types: the menu

Role maps to behavior, context to scope, tone to communication, format to structure — and the mapping keeps going. The menu below names fourteen requirement types; you pick from it, you never fill it. Each type carries acceptance evidence — how to check compliance — in the requirements catalog, printed in full in references/Elements_of_Prompting_Field_Guide.pdf.

What the work is and who it serves

- **Functional behavior** — what it must do: filter, calculate, extract, compare, explain. Weak: "analyze the returns data." Strong: "calculate the return rate per supplier and rank all three."
- **Scope and exclusions** — in, out, explicitly outside. Weak: "use the relevant data." Strong: "detail rows only — exclude the TOTAL row; keep defects separate from returns."

- **Audience and use** — who uses it, for which decision. Weak: "make it useful for leadership." Strong: "for a manager deciding whether supplier follow-up is warranted."
- **Role and working behavior** — perspective plus observable conduct. Weak: "act as a reviewer." Strong: "act as a reviewer: expose assumptions and gaps."

What goes in and how it is computed

- **Data and provenance** — allowed inputs, definitions, period, traceability. Weak: "use the supplier data." Strong: "this workbook's Data tab, 2025-09 to 2026-08 inclusive; trace each figure to its rows."
- **Method and business rules** — formulas, denominators, missing-data treatment. Weak: "use our standard definitions." Strong: "return rate = total returns ÷ total units shipped, from sums, never averaged percentages; the blank Inspection_Hours cell is missing, not zero."
- **Content and completeness** — required topics, fields, limitations. Weak: "cover the important points." Strong: "the business question, the comparison, one time view, a follow-up, limitations."

How the output looks and behaves

- **Structure and interface** — sections, tabs, slides, columns, controls. Weak: "make it a nice table." Strong: "one row per supplier: Supplier | Return rate | Numerator | Denominator — highest rate first."
- **Tone and presentation** — voice, reading level, units, number formats. Weak: "keep it short and professional." Strong: "neutral, plain language for managers; rates to three decimal places."
- **Accessibility and usability** — labels, contrast, keyboard access, understandable language. Weak: "make it user-friendly." Strong: "labeled controls, visible focus, readable contrast, a usable narrow layout."

How the result is checked, bounded, and kept

- **Quality and acceptance** — observable correctness criteria. Weak: "double-check your work." Strong: "reconcile total shipped and returned against the TOTAL row before presenting anything."
- **Constraints and authority** — allowed tools, approvals, stop conditions. Weak: "be careful with the emails." Strong: "prepare the reply as a draft for review; do not send it." (Prompt instructions request compliance; actual permissions determine what a tool can do.)
- **Delivery and compatibility** — file type, editability, offline use, destination. Weak: "send me a dashboard." Strong: "one self-contained HTML file that works offline, data embedded, no network calls."
- **Maintenance and reuse** — refresh procedure, versions, reusable parameters. Weak: "so we can use it again." Strong: "label the file a static data snapshot with its date; keep reusable placeholders."

Examples, uncertainty, and stopping

Examples are the strongest steering signal for format and tone: the model imitates what it sees, and on well-tuned models examples steer shape more than facts (Brown et al., 2020; Min et al., 2022). The discipline is diversity: three clones of the happy case teach only the center; one typical, one edge, and one reject case — clearly separated — teach the boundaries too. Use an edge case when it exposes a likely failure, and never smuggle obsolete facts in with the sample.

The Out is permission to not know, in checkable shape. A model is trained to always produce an answer; unless you name the honest move, inventing is the only way it can obey you. One sentence of permission cut invented answers on planted-error questions from 53 to 23 percent on GPT-4o (six-model average 66 to 44 — Omar et al., 2025), and to under 8 percent on GPT-5 (npj Digital Medicine, 2026). "Write Not stated and list the gap" is the pattern — a named form you can look for in the output.

TRAP — Bare bans backfire

"Do not hallucinate" names no honest alternative, and a ban alone can trigger over-refusal of facts that are in the supplied material. Say what to do instead: the Out, plus required citations — it helps without guaranteeing that nothing is invented.

The Stop is the finish line: the action to perform and its boundary. Weak: "keep it focused." Strong: "answer these three questions, report limitations, then stop - do not go exploring." A completion condition turns open-ended capability into a scoped job - an answer instead of ten charts. In agentic work this valve grows up to become gates and autonomy rules.

KEY — The unifying idea

Each element moves something out of your head onto the page. When output disappoints, one of them stayed in your head — and the diagnosis grid below tells you which.

A menu of requirements for each artifact

The artifact decides which requirement types matter most. The working method: recognize the artifact, choose the types that resolve ambiguity in this task, then write a concrete instruction and name the evidence that would show it was met. Chapter 4 works through nine artifacts and closes with a table of the types that dominate each.

The per-artifact tables — requirement type, example instruction, how to check it — are printed in the field guide; the same menus drive the Prompt Template Creator ([deliverables/Prompt_Template_Creator.html](#)) and the Configurator ([references/Prompt_Template_Configurator.xlsx](#)). Treat it as a catalog: something disappointed, name the weak spot, pick a stronger fill. You do not study it; you look things up in it.

The improvement loop

A first prompt is a draft, and the way from draft to dependable is a loop you may already run on processes: Plan, Do, Check, Act — the improvement cycle of W. Edwards Deming, which Toyota built its production system around. The Lean Enterprise Institute published a piece in May 2026 applying it to prompting as "Prompt, Do, Check, Act"; their observation: the people who get the most from these tools are the ones willing to run the loop a few more times. Every phase can be partly delegated while judgment stays with you.

- **Plan — draft the ask.** Fill the elements the task needs; say why behind each rule; separate instructions from pasted material; show examples, one edge case included. Delegate it: "I need [X] for [audience], with [constraints]. Draft me a strong prompt, then wait."
- **Do — run it, in order.** Number the steps; add "wait for my OK" where you want control; documents on top, ask at the end; fresh chat per topic.

- **Check — inspect before you trust.** Name the failed element with the grid below; self-check against named criteria — never "are you sure?"; blind-review by pasting the draft as "a colleague's"; reconcile numbers to a total you already trust.
- **Act — refine, then standardize.** Fix one element and rerun the affected check — never reword at random; when a prompt passes representative checks, name it, version it, save it; re-baseline on every model upgrade. Delegate it: "Rewrite this prompt so it more consistently produces X."

The output is...	The element that failed
the wrong altitude, tone, or posture	Role
an answer to a different (or vaguer) question	Task
generically right, specifically wrong for us	Context
right content, unusable shape or length	Format
not matching the standard in your head	Examples
confidently invented	The Out (missing)
sprawling past what you asked	The Stop (missing)

The symptom-to-element diagnosis grid — also in the EX-Report tab of deliverables/Course_Workbook.xlsx. Name the failed element, fix that one, rerun.

Keep a change only when it improves the result on representative examples, not one lucky run. Model self-review helps find issues, but it is not proof — the checks that count compare the output with evidence or a known calculation.

One task through the improvement loop

The loop at its smallest, on the supplier case. Initial ask: "Summarize supplier returns." Observed problem: the total includes a pre-calculated TOTAL row — units counted twice. Revision, aimed at that failure: "Use detail rows only; exclude the TOTAL row." Verification: compare with the source total — an independent check only when scope, filters, and units match. Change the instruction linked to the failure; keep the business question stable. A passing test supports reuse; it does not establish universal reliability.

Run enough turns and the one-line ask grows into the full analysis prompt - each added line a requirement type fixing one named weakness. (Expected output: Chapter 4, step 1; self-check: Appendix B.1.)

Use Supplier_Data_Exercise.xlsx, Data sheet, detail rows only – exclude the TOTAL row (144 rows; one Inspection_Hours cell is blank: treat it as missing, not zero, and disclose it). One row = one month × site × supplier; period 2025-09 to 2026-08 inclusive. Question: which supplier has the highest RETURN RATE, defined as total Units_Returned ÷ total Units_Shipped per supplier over the full period – computed from sums, never by averaging monthly percentages. Keep Defects_Found separate from returns; do not add them together. Rank the three suppliers showing numerator and denominator, reconcile total shipped and returned against the TOTAL row, and state one limitation of the comparison. Then stop.

The fully specified form of the tab 1-Analyze-Data prompts - every line names one requirement type. This is the ASK form reused in Chapter 5.

One clarification: the anatomy is the **design layer** — what a prompt contains; is it complete? The loop is the **process layer** — what you do with it; how does it get, and stay, good? In chat you run the loop by hand between turns; in agentic work you write the loop into the brief itself, which is why it works best there:

PLAN Role: careful retail analyst – never invent numbers. Task: monthly returns report from the attached export. Context: columns = SKU (product code) · reason code · refund €. Format: one-page summary + top-5 table.
 Example: last month's report, attached. The out: field missing → say so, don't guess. The stop: deliver the report – nothing beyond.
 DO 1) Read the export; list the columns you actually see. 2) WAIT for my OK.
 3) Build the table. 4) Draft the summary.
 CHECK Before showing me anything: reconcile the table to the export TOTAL.
 If it doesn't reconcile, STOP and report the gap.
 ACT Deliver the final report. Then list what was ambiguous in this brief, so I can fix it for next month.

The loop written into one brief: a wait-gate is a Do you kept; a stop-on-fail Check is a verification you refused to delegate.

KEY — Two layers, one craft

You write with the anatomy; you improve with the loop. And Act does both of its jobs: deliver the output, and name what to improve — the prompt asks for its own next revision.

Your turn: turn your course log into a report

If you have been running this book's exercises in one chat - the course log Prompt 1 set up - close the loop by turning that log into a report. Two prompts, in that chat:

STEP 1: List every prompt I have run in this course log, in order – one line each: what it did, and which element or move it taught.

STEP 2: Now turn that into my one-page course report: the seven elements with my own example for each, the loop, and the three prompts I will reuse at work. Anything the log doesn't show: write UNKNOWN – don't invent. Deliver the report, then stop.

Tab EX-Report of Course_Workbook.xlsx - run both in the course log you opened with Prompt 1. Reference Deck slide 104 holds a one-prompt variant: a study aid listing concepts, one example each, the checks used, and two open questions.

Notice the craft inside step 2: named content requirements, an Out ("anything the log doesn't show: write UNKNOWN") and a Stop ("deliver the report, then stop") - the course's own rules applied to the course. The report is your first library entry: name it, date it, keep it. If the log was lost, the numbered prompts in the workbook tabs are the full list - rebuild from there.

CHECK — What a generated summary proves

Generated reflection supports recall; it does not prove skill. The demonstration that you can do the work is the capstone task, checked against its own acceptance evidence.

The prompt is a document, not a sentence

A prompt that has been through the improvement loop a few times stops looking like a sentence and starts looking like a one-page document with named parts. That is the mature form of the work specification, and the course ships a worked template for the data-analysis case: five labeled parts, each with a reason it exists.

```
<role> Careful analyst: compute by RUNNING CODE - never mental math. State n. Never drop data silently.
<data> File · sheet · header row · what one row means · exact column names · known quirks · metric definitions.
<task> Numbered questions in priority order - "and stop when they're answered."
<method> Profile first and show me · reconcile to a known total · flag small groups and explain the limitation · "associated with", never "caused by".
<format> Headline answer with the number FIRST, then the table; assumptions and caveats at the end.
```

The template skeleton. The full version with blanks marked is in workbook tab G2-DataAnalysis - copy-paste, don't retype.

Part	Why it is written this way
Role - code, not vibes	Language-model arithmetic is unreliable; code makes the computation inspectable - and the formula still needs checking through code, spreadsheet formulas, or an independent calculation.
Data - schema first	The model guessing your row grain and denominator is an important source of wrong answers.
Task - questions, not topics	Topics generate exploration; questions generate answers. The "and stop" prevents ten charts and no answer.
Method - one anchor	Reconciliation checks the totals; other errors may require other checks - and the language discipline ("associated with", never "caused by") survives into the deck leadership reads.
Format - answer first	You read the answer in ten seconds and audit the rest only if it matters.

KEY — Documents are reusable; sentences are not
Every prompt in this course lives in a named workbook tab for exactly this reason: a one-page document with labeled parts can be copied, filled, reviewed, and improved. A clever sentence cannot.

Prompt techniques to test

Treat every technique as a hypothesis to test on representative examples: research effects depend on the model, task, data, and measurement, and one benchmark is not a universal effect size. Still, the evidence sorts advice into three piles.

WORKS — Replicated — reliably helps
Specificity, grounding, structure, and named checks survive replication across models — the elements of this chapter, measured.

- Be specific - task, constraints, success criteria: the unstated-requirement gap from Chapter 1.
- One tested, reused template with delimiters (labeled sections or tags): formatting alone can swing accuracy up to 76 points (Sclar et al., 2024).
- Long inputs: documents on top, question at the end (Chapter 1).
- An out in checkable shape, plus required citations: invented answers cut by a third or more.

- Self-check against named criteria: fact scores jumped 56 to 71 with planned verification (Chain-of-Verification, 2024).
- Ask the assistant to improve your prompt: optimizer-written prompts beat human ones; every major vendor ships an official improver.

MYTH — Never survived replication

Flattery, threats, and magic phrases move style at best and add noise at worst. "The AI knows what I mean" is the same 41 percent from the other side — say what you mean is the cure.

- Expert personas for accuracy: zero gain across 162 personas.
- Tips, threats, "important to my career": null on average; the famous gain was a cherry-picked best case.
- Magic phrases ("take a deep breath"): each won on one model, one benchmark - never universally.
- "Longer prompts are better": padding drops reasoning accuracy at around 3,000 filler tokens - density wins.
- "Are you sure?" as verification: answers flip under pushback (Chapter 1).

EXPIRED — Was right — then the product absorbed it

"Think step by step" was real — one math benchmark went from 10 to 41 percent in 2022 — until vendors built the stepping into reasoning models. Advice expires when the product absorbs it.

- Manual chain-of-thought: pick a reasoning model and set the effort level (still legitimate on small local models).
- Piles of examples: start zero-shot (no examples); add three to five format-definers only when shape drifts.
- Hand-run voting over several samples and micro-decomposed steps: absorbed into effort settings; chain steps only where you need auditable intermediates.
- "You are a world-class expert" for accuracy: persona for voice only.
- Carrying an old prompt stack onto each model: re-baseline on every upgrade - strip to the anatomy, re-test, re-add what earns its place.

The full lists, with the study behind every line, are in the PLAYBOOK tab of deliverables/Course_Workbook.xlsx; the field guide's evidence compendium carries the why per verdict. What holds throughout: clarity, context, format, the Out, grounding, iteration — scaffolding tricks retire; briefing skills compound.

A practical prompt review checklist

With the mirror named (Chapter 1) and the loop in hand, the chapter compresses into a review checklist. **Clarity:** one identifiable task, explicit definitions, a useful output. **Relevance:** only instructions and source material that help the task. **Boundaries:** missing-information behavior, a completion condition, authority limits where actions are involved. **Evidence:** a source check, a calculation check, or an observable acceptance test. Test advice in your actual workflow - more words, stronger praise, or a magic phrase cannot guarantee a better result.

PART 4 - PUTTING PROMPTS TO WORK

Putting prompts to work: one dataset, nine steps

Everything so far has been about writing requirements. This part puts them to work on one running case: a fictional supplier-quality question. One dataset is uploaded once and carried through nine steps - from a checked analysis to a research workbook - with a plain-language explanation as the reference exercise. The data, the prompts, and the checked answers all ship with the course, so you can run every step and verify every number you produce.

One dataset, nine steps

The question is deliberately ordinary: which supplier has the highest return rate, and what should we investigate next? A manufacturer receives components from three fictional suppliers - Alpha Components, Bravo Plastics, and Cardinal Metals - at four sites (Austin, Berlin, Osaka, Monterrey), and the practice workbook records twelve months of deliveries, September 2025 through August 2026. All names and numbers are generated for training; nothing in the case is real company data, so you can upload it wherever your policies allow.

The first rule is set before any prompt: the whole data arc works from ONE dataset, uploaded once at the very beginning. `Supplier_Data_Exercise.xlsx` goes into the chat at intake, and everything through the management mock-up flows from that single upload. Then the case widens - an email thread, three quotation documents, and a pack of reference files each bring their own source material, but the discipline stays the same.

Step	What you make	Where
1 · Inspect, then analyze	A checked supplier comparison, reconciled to the source	Slide 37 · tab 1-Analyze-Data
2 · Follow up in the same chat	A month-and-site pattern - values shown, no causes claimed	Slide 38 · tab 1-Analyze-Data
3 · The assistant returns your workbook	<code>Supplier_Data_Analyzed.xlsx</code> - Summary sheet, native charts	Slide 39 · tab Excel-Charts
4 · Build the dashboard	One self-contained offline HTML file from the returned workbook	Slides 40-41 · tab 2-Build-Dashboard
5 · Present the findings	A five-slide management mock-up from the same verified findings	Slide 42 · tab 3-Present-Findings
6 · Organize the email thread	The structured brief - decisions, actions, "Not stated"	Slides 43-45 · tab 4-Summarize-Email
7 · Extract the quotes	A Raw Extraction sheet - verbatim values, cited sources	Slide 47 · tab Quote-Extract
8 · Compare the quotes	A Normalized Comparison - or an honest "not yet"	Slide 48 · tab Quote-Compare
9 · Build the research workbook	Evidence · Synthesis · Sources, every claim traced	Reference Deck slide 112 · tab Research

Slide numbers are those of `From_Prompts_to_Agents_Facilitated_60_Minute.pptx` (slides 43 and 46 open the email and document blocks); the reference exercise (explain a topic clearly) is Reference Deck slide 102 · tab 5-Explain-Clearly.

Why one case instead of nine unrelated ones? Because the requirement types recur and the artifact decides which ones matter; watching the same facts pass through nine shapes makes that pattern visible, and the table at the end of this chapter collects it.

A basic prompt, and what it decides for you

Analyze the supplier data and tell me which supplier is worst.

The starter prompt. Run it once before improving anything - then compare it with what follows.

This prompt runs, and it returns something confident. But it made three decisions silently - the rows, the metric, and the meaning of "worst", the gaps Chapter 3 named - and it carries no source scope, no denominator rule, no missing-data rule, and no check. Every gap becomes the model's guess - and a guess you cannot see is a guess you cannot audit.

KEY — Write the requirement, not the wish

A prompt improves by adding the requirement that fixes a specific observed weakness. Each added line names one gap and closes it; nothing is added for decoration.

Step 1: inspect, then analyze

The live exercise starts with exactly one file open:

references/exercise-data/Supplier_Data_Exercise.xlsx, a workbook whose single Data sheet holds the case records. One row is one month-site-supplier observation. Units_Shipped counts units received that month; Units_Returned counts units customers later returned for component defects - the escapes; Defects_Found counts defective units caught at incoming inspection. Keeping those last two apart is the case's first discipline. Inspection_Hours, Unit_Cost_USD, and On_Time_Percent complete the row; On_Time_Percent is a monthly percentage, a detail that matters later. The workbook's README tab defines every column - a ready-made data block you can paste into a prompt rather than retype.

Two quirks are planted on purpose: the last row of the Data sheet is a TOTAL row that must be excluded from any calculation, and one Inspection_Hours cell is simply blank. A good first prompt finds both - which is exactly why the task starts with a profile, not a question.

KEY — Profile first, analyze second

Inspect the data and its boundaries before asking for conclusions. Make the model look at the file and report what it sees; only then ask your question.

Before calculating anything, inspect the file. Tell me what one row represents, how many detail rows it contains, whether any values are missing, and whether any total or summary row could be counted twice. Tell me what you would include or exclude, then wait.

The inspection prompt, verbatim - workbook tab 1-Analyze-Data. "Then wait" is the working boundary: no analysis until the profile is confirmed.

CHECK — Success check

A good profile reports 144 detail rows, one TOTAL row to exclude, and one missing inspection-hours value. If it missed a quirk, tell it and make it re-profile - that correction is what the step is for.

The two quirks need different treatment. The TOTAL row is redundant information: including it double counts every value, so it is excluded - but kept nearby, because it becomes the reconciliation anchor in a moment. The blank cell is absent information: it must stay missing, never become zero, and any calculation that excludes it must say so. Exclusion and disclosure are different obligations.

The checked analysis

Using detail rows only, calculate each supplier's return rate as total Units_Returned divided by total Units_Shipped. Rank suppliers from highest to lowest. Show the totals used, reconcile the overall totals to the source file, and state one limitation of the comparison.

The analysis prompt, verbatim - workbook tab 1-Analyze-Data.

The result is checkable to the digit:

Supplier	Units shipped	Returns	Return rate
Bravo Plastics	75,184	262	0.348%
Cardinal Metals	74,658	96	0.129%
Alpha Components	75,060	74	0.099%
All suppliers	224,902	432	0.192%

Defects found - 2,207 occurrences caught at incoming inspection - are a separate measure. Their sum with returns is not a count of distinct defective units.

Bravo Plastics has the highest return rate, roughly three and a half times Alpha's, and the shipped and returned sums reconcile exactly against the TOTAL row. A defensible limitation to state: the data show which supplier has more returns, not why.

KEY — Confidence is not the deliverable

Correct calculations, honest limits, and reconciliation matter more than confident prose. A ranked table with numerator, denominator, one reconciliation, and one stated limitation is a complete answer.

Common wrong turns

TRAP — Five ways to be confidently wrong

Each of these produces a plausible-looking answer. The improved prompt forbids all five explicitly, and the acceptance checks below catch them when it fails to.

- Averaging the monthly return-rate percentages instead of dividing summed returns by summed shipments - months with different volumes get equal weight, so the denominator is wrong.
- Adding Defects_Found to Units_Returned and calling the sum "defective units" - inspection catches and customer returns are different measures, and their sum is not a count of distinct defective units.
- Treating the blank Inspection_Hours cell as zero - a missing measurement is not a measurement of nothing.
- Including the TOTAL row, which double counts every column it summarizes.
- Explaining the result - "Bravo's process causes the returns" - when the data supports association at most, never a cause.

Acceptance checks

- Row count: exactly 144 detail rows in scope after excluding the TOTAL row.
- Reconciliation: summed shipments and returns match the TOTAL row exactly - 224,902 units and 432 returns.
- Traceability: the displayed 0.348% recomputes from its parts, 262 returns over 75,184 units.
- Disclosure: the missing Inspection_Hours value is named wherever it affects a calculation.
- Limits: one limitation is stated, and comparative language stays at association, not cause.

Grade your own run against Appendix B.1 - after attempting, not before.

Step 2: one follow-up, in the same chat

The second step costs one sentence, because it spends context the first step already paid for. The chat that ran the inspection and the analysis knows the file, the row grain, the exclusions, and the metric definitions - so a follow-up question inherits all of it. Starting a fresh chat would mean re-earning that scope; asking in the same chat keeps it pinned.

Using the same data, compare return rate by month and site. Identify the clearest increase, decrease, or unusual pattern. Show the values behind it. Do not claim a cause. List the additional data needed to investigate why it happened.

The follow-up prompt, verbatim - workbook tab 1-Analyze-Data.

This too has a checkable answer. Bravo's monthly return rate is elevated and volatile rather than steadily worsening: it peaks near 0.737% in April 2026, drops to roughly 0.14% in the quiet months of June and July 2026, and sits near 0.465% in August 2026. "Volatile, with no steady improvement" passes. Any claimed cause fails, because nothing in the file supports one.

KEY — Patterns yes, causes no

The prompt asks for the values behind the pattern and the additional data an investigation would need - and explicitly forbids a causal claim. The file shows which supplier has more returns and when; it cannot show why.

A related trap, if you also ask about on-time delivery: On_Time_Percent is a monthly percentage with no delivery counts behind it, so no overall on-time rate can be computed from this file. Label any average as an unweighted mean of monthly percentages, say plainly that it is not a delivery rate, and name the missing data - deliveries per month. Appendix B.1 has the values.

Step 3: the assistant returns your workbook

The third step changes the deliverable from an answer to a file. Modern assistants can edit the workbook you uploaded and hand it back - and that turns the prompt into a file contract: what the returned file is named, what it must contain, and what must survive untouched.

Using the verified calculations, update the uploaded workbook and return an editable Excel file named `Supplier_Data_Analyzed.xlsx`.

Add a Summary sheet with:

- A supplier comparison table showing units shipped, units returned, return rate, and defect rate.
- An editable native Excel column chart comparing supplier return rates.
- An editable native Excel line chart showing return rate by month.
- The reporting period, metric definitions, and one short limitations note.

Keep the original data unchanged. Use formulas, PivotTables, or other traceable Excel logic rather than hard-coded chart values. Preserve missing values as missing. Check that the summary totals reconcile to the source data before returning the file.

The Excel analysis-and-charts prompt, verbatim - workbook tab Excel-Charts.

When the file comes back, open it like an auditor. The Data sheet must be unchanged and the missing value still missing. The Summary must reconcile to the Data totals through formulas you can click, not pasted numbers. And the two charts must be ones you can retitle and recolor - if you cannot edit a chart, it is a picture, and pictures fail the contract.

CHECK — Editable-native is the acceptance test

Summary totals reconcile to the Data sheet totals, with a reconciliation check on the sheet itself, and both charts respond to editing. A returned file that fails either test goes back with the failure named.

The prepared fallback ships in `references/exercise-data/`: `Supplier_Data_Analyzed.xlsx`, the returned workbook this exact prompt produced. It matters beyond this step - the dashboard and the presentation both take it as their input, so the metric definitions verified here carry forward instead of being re-derived.

Step 4: a dashboard from the returned workbook

The fourth step turns the analysis into a tool. The input is not a fresh export - it is the returned `Supplier_Data_Analyzed.xlsx` from step 3, checked definitions and all, so the dashboard can never quietly disagree with the analysis about what a return rate is.

KEY — You specify behavior, not code

An HTML dashboard is a web page: the file provides the structure, and JavaScript - the browser's scripting language, embedded in the same file - supplies the interactive behavior. You specify that behavior in plain language, control by control - no code knowledge is needed to ask for it precisely.

Using the returned `Supplier_Data_Analyzed.xlsx` workbook and its verified definitions, create one self-contained HTML dashboard that works offline. Include a date-range selector, supplier and site filters, a metric selector, summary values, a trend chart, a sortable detail table, and a reset control. Every filter must update every view. Show the selected period and metric definitions. Display a clear message when no rows match.

The dashboard prompt, verbatim - workbook tab 2-Build-Dashboard.

What makes a KPI worth a tile

A KPI (key performance indicator) is a number someone acts on, not decoration. Before you approve a tile row, hold each candidate against four tests:

- A decision. If nobody would do anything differently when the number moves, it is trivia, not a KPI - cut it.
- A formula. Named columns and exact arithmetic; if you cannot write the formula, you cannot track the number.
- A target. A value is a fact; value-versus-target is a status.
- Three views. Every KPI answers level (where are we), trend (which way is it moving), and gap (how far from target). Tiles show level and gap; the chart shows trend.

Name the parts and you can ask for them

Each control earns its place by answering a business question. The month range and the granularity switch answer when did it change? The supplier and site filters and the comparison switch answer who is different? The metric selector answers different on what? Drill-down answers which records back this number? Reset makes exploration safe. Controls are questions, not decoration - add one only when its question matters. A few vocabulary essentials carry most requests:

Term	What it is	Ask for it like this
KPI tile	Big-number card: value, vs-target state, trend arrow	"A KPI row of four tiles across the top."
Delta	Change vs the prior period, beside the value	"Each tile shows the delta vs last month."
Sparkline	Tiny axis-less trend line inside a tile	"Add a 12-month sparkline inside each tile."
Ranked bars	Bar chart sorted by value - who is worst	"Ranked bars of return rate by site, worst first."
Heatmap	Grid colored by value, two dimensions at once	"A site-by-month heatmap of on-time %."
Filter / slicer	One control that narrows every view at once	"Site and Supplier filters that apply to all views."
Drill-down	From a chart segment to its underlying rows	"Click a segment to show its records."
As-of date	When the data was last refreshed, on the page	"Footer: as-of date, source, owner, refresh cadence."

The full twenty-term vocabulary with ready-made phrasings is in the workbook's EX-Dashboard tab. It is the transferable part - the same words work in Power BI and Tableau conversations.

Empty, missing, and zero are three different facts

An empty selection must say "no matching records" – distinguish it from a real zero. Missing values stay blank; a zero denominator shows "—", never an error.

The states requirement, verbatim - workbook tab 2-Build-Dashboard.

An empty selection means no records matched; a zero means records matched and the value really is zero; a blank means the value was never measured. A dashboard that collapses these into one symbol will eventually be read wrong in exactly the meeting where it matters. The zero-denominator dash prevents the ugliest failure of all: a division error styled as a number.

The check that makes it trustworthy

Filter to Site = Berlin, Supplier = Bravo Plastics, months 2026-03 to 2026-08. Expect exactly 6 records, 8,575 units shipped, 21 returns, return rate 0.245% – in the tiles, the chart, AND the table. Verify independently by filtering the workbook the same way.

The acceptance check, verbatim - workbook tab 2-Build-Dashboard.

This is the coordinated-views requirement made concrete: one selection re-scopes every view, so the tiles, the chart, and the table can never disagree about which records are on screen. One verified filtered result beats any amount of admiring the design. The fuller dashboard menu, each type with its acceptance evidence, is in references/Requirements_by_Artifact.md.

The prepared fallback, and two cautions

A finished, checked build ships in the package:

references/exercise-data/Supplier_Quality_Dashboard.html, built from the same records and metric definitions as the analyzed workbook. Open it offline, run the Berlin filter yourself, and read the footer: it carries the definitions, the snapshot date, and a self-check that runs this same filtered comparison - it must say PASSED. If your own build stalls, the fallback lets you study working behavior first and rebuild later.

Building the file yourself needs an assistant that writes and packages code into a downloadable HTML file. As of September 2026 that means Claude (Artifacts), ChatGPT (Canvas), or Gemini (Canvas), typically on paid tiers; in Copilot it means Pages, not plain chat. If your tool only chats, the specification still has value - hand it to IT as written, or ask for an Excel version of the same views.

TRAP — A snapshot, not a system

The file embeds its data, so classify and share it like the spreadsheet it came from - share the file, not a public link. And label it what it is: a static data snapshot with its date, not a live reporting system. Refreshing data is a job for a business-intelligence (BI) tool such as Power BI or Tableau.

Step 5: five slides from the same findings

The supplier case now changes audience. Your verified analysis answered the analyst's question; a management audience does not want to see the analyst's screen. It wants a decision framed, evidence it can trust, and a visible boundary between what the data show and what you propose to do about it. A presentation is a deliverable with its own artifact requirements, and writing the prompt means choosing them deliberately.

For a management deck, eight requirement types carry the weight. **Purpose and audience:** name the decision and the intended action. **Scope and slide count:** five slides with a defined coverage list. **Story structure:** business question, evidence, implications, next action. **Message hierarchy:** one main message per slide. **Data fidelity:** only checked findings, with period, units, and limitations preserved. **Chart semantics:** truthful axes, labeled categories. **Speaker notes. Editability and delivery:** an editable .pptx you can actually present. The full menu, with an acceptance check for each type, is the presentations table in references/Requirements_by_Artifact.md.

Using only the verified findings from this exercise, create an editable five-slide PowerPoint mock-up for [audience] to support [decision]:

1. Question and scope.
2. Supplier comparison.
3. The most important trend.
4. Recommended follow-up.
5. Limitations and next steps.

Use one main message per slide, readable charts, concise visible text, and speaker notes. Separate findings from recommendations. Do not invent causes or commitments.

The presentation prompt, verbatim - workbook tab 3-Present-Findings.

The mock deck, slide by slide

The prepared output this prompt produced is `Supplier_Quality_Mock_Presentation.pptx` in `references/exercise-data/` - a fictional five-slide management deck built only from the verified analysis, in a deliberately distinct navy look so it never reads as part of the course deck. Open it beside `Course_Workbook.xlsx`; every number on it traces to the workbook.

- Question and scope - one question, one period (2025-09 to 2026-08), four sites, three suppliers, 144 delivery records, one definition of return rate for every comparison. The decision the deck supports is named on the title slide: is a supplier follow-up warranted?
- The comparison - a bar chart with the three rates beside it, and a side note that closes the obvious objection: shipment volumes are nearly equal, so the rate gap is not a volume artifact.
- The trend - Bravo's twelve monthly rates: elevated and volatile, no steady improvement - and no claim of seasonality or cause.
- The proposed follow-up - a two-panel slide. Left, labeled findings: the rate gap, its persistence, and the separate fact that Bravo drives 1,157 of the 2,207 inspection defect occurrences. Right, labeled judgment: ask Bravo for a returns root-cause review, pull the records behind the worst months, agree a target and re-measure next quarter on the same definition.
- Limitations and next steps - descriptive, not causal; no delivery counts, so no delivery rate is claimed; one inspection-hours value missing, not zero; inspection defects and customer returns never added together. Then the single decision requested: approve the review.

The separation rule

Findings are what the workbook supports; recommendations are labeled proposals, and the two-panel slide makes that boundary physical - the trust move that survives every audience question. It matters most exactly where the story gets tempting. The verified scorecard carries units-weighted unit costs: Alpha Components \$12.4299, Bravo Plastics \$9.9030, Cardinal Metals \$14.1804. The tension is real - the cheapest supplier is also the most-returned - and it is precisely where a weak deck starts inventing: a projected savings figure, a quality-cost trade-off, a committed benefit. The mock deck states no cost claim anywhere, because the data cannot support one; what switching or fixing would cost is a question for the proposed review, not for this dataset.

TRAP — The invented benefit

A decision audience remembers the number you made up, not the caveat you attached to it. Bravo's low unit cost invites a savings story the workbook cannot back - leave it out.

Check it like an artifact

- Coverage: count five slides and map each one to the coverage list in the prompt.
- Fidelity: reconcile every displayed number to the analysis - at minimum trace the 0.348% comparison and the April 2026 peak of 0.737% back to the workbook.
- Narrative: read only the slide titles. Do they tell the story alone, from question to evidence to proposal to limits?
- Delivery: open the exported file and inspect it - no clipped text, readable at projection size, speaker notes present on every slide, charts still editable.

TRY IT — Audit before reuse

Run the four checks on `Supplier_Quality_Mock_Presentation.pptx` before you reuse the prompt on your own analysis. The sample is deliberately auditable - the limitations slide names what the data cannot say, which is the habit to copy.

Step 6: organize the email thread

An email summary has a job to do. A long thread read top to bottom is a chronology, and a chronological retelling can bury the current decision - the thing the person asking for the summary actually needs. So decide what the summary must answer before writing any prompt; the menu below names the jobs.

Choose the result you need

One thread can serve five different jobs, and choosing the job IS the prompt - one sentence each once the outcome is named. In Outlook the workflow is: run the built-in Copilot summary first, then refine it in the Copilot chat pane, scoped to the same conversation, with the line that matches what you need.

You need	Say
Catch me up	"Summarize the current situation in three sentences."
What changed	"List changed dates or decisions and show the latest version."
Who owes what	"Create a table with task, owner, due date, and dependency."
What remains open	"List unanswered questions, approval conditions, and missing attachments."
Prepare my reply	"Draft a concise response. Use brackets where I still need to decide. Do not invent commitments."

The five output shapes, named

Behind the menu sit five reusable output shapes. Naming the shape is what turns "summarize this" into a specification - each shape has a job, and blank cells beat guessed ones in every one of them.

Shape	What it is - and when to use it
1 · TL;DR triage	One sentence, ~30 words: current state or decision needed - not the history. For clearing an inbox.
2 · The structured brief	OVERVIEW · DECISIONS · ACTION ITEMS · OPEN QUESTIONS. The default when filing, forwarding, or catching up - the prompt below is this shape, grown up.
3 · Action-items table	Task Owner Due Blocked by. For moving work into your to-do system.

Shape	What it is - and when to use it
4 · Decisions log	Decision Decided by Reasoning Date - settled items only, proposals flagged. For defending choices later.
5 · "Who owes what"	Every unanswered question + "X owes Y: [thing]", latest state only. For preparing your reply.

The requirement types for this artifact: **source scope**, **current state**, **decisions and conditions**, **actions and ownership**, **date meaning**, **evidence traceability**, **attachments**, and the **authority boundary**. Chronology and evidence dominate: a useful summary preserves who said what and when, keeps every condition attached to its approval, and traces each claim to a supporting message. The email-summaries table in references/Requirements_by_Artifact.md pairs each type with its acceptance check.

Review this thread using only the messages and attachments you can access. Give me the current status, confirmed decisions and conditions, action items with owner and due date, changed dates or commitments, unanswered questions, missing attachments, and messages that need a reply. Write 'Not stated' when the thread does not say. Cite the supporting message by sender and date.

The structured brief, verbatim - workbook tab 4-Summarize-Email.

Notice the honesty machinery. "Not stated" beats a guessed owner or date; a citation by sender and date makes every claim checkable; and the brief only organizes - nothing is sent. The fuller template - decisions and actions as tables, superseded dates marked as superseded, the next clarification requested, a reply drafted only for review - is the library-entry example in Chapter 6.

The practice thread

The practice material is a fictional packaging-change conversation - ten messages, six people - printed in full in Appendix A.2 and supplied as Packaging_Change_Thread.txt in references/exercise-data/, so you can paste it into any approved assistant. The attachment mentioned in message 5 is intentionally not supplied.

The four traps

The thread is engineered so that a naive "summarize this" fails in observable ways. Four traps are planted - two moving dates, a conditional approval, an unanswered question, and a missing attachment - and a good brief catches all four.

CHECK — Success check

Your brief catches all four planted traps - and gives Ben no invented due date. The expected brief, item by item, is Appendix B.2.

Where to run it

The pasted-text path is primary: paste Packaging_Change_Thread.txt and the prompt into any approved assistant; it always works and needs no particular license. The native path, verified as of September 2026: Microsoft Outlook's "Summary by Copilot" on an open thread, with numbered citations that click through to the source message. The citations are your verification tool. Then run the same prompt in the Copilot chat pane. That depends on your organization's Copilot license, which is why the course drills the pasted version. Either way the prompt is portable: scope, extraction fields, evidence rules, and authority boundaries travel with you.

Draft my reply for review: answer what the thread can answer, confirm decisions WITH their conditions, and propose one next step per unresolved item. Leave [brackets] where I must decide. Do not send anything.

The optional follow-up - a draft is the boundary; sending stays human.

TRAP — Practice on fiction only

Never practice on real confidential threads. The packaging conversation exists so you can drill extraction on material that is safe to paste anywhere; run your first summaries on it, not on live mail.

Step 7: extract the quotes

After the email arc the case turns commercial. Three fictional quotation documents - Quote_Alpha_Components.pdf, Quote_Bravo_Plastics.pdf, and Quote_Cardinal_Metals.pdf in references/exercise-data/ - answer the follow-up the analysis proposed. The job is documents in, spreadsheet out, and it is deliberately split into two steps: faithful extraction first, judgment second.

Review the attached supplier quotation files and create an editable Excel workbook.

Create a Raw Extraction sheet with one row per quotation and these fields: supplier, quoted scope, currency, quoted quantity, unit price, tooling or other one-time charges, freight, taxes, lead time, payment terms, quote validity, exclusions, source file, source page, and missing information.

Copy values exactly as stated. Keep currencies, quantities, and units in their original form. Write 'Not stated' when a field is absent. Do not normalize, rank, or recommend yet. Cite the source file and page for every extracted commercial value.

The extraction prompt, verbatim - workbook tab Quote-Extract.

Every discipline in that prompt exists because its absence has a failure mode. Verbatim values, because a "helpfully" rounded price cannot be audited. Original currencies and units, because silent conversion smuggles in an unstated exchange rate. "Not stated" for absent fields, because a guessed freight term looks exactly like a real one. And a source file and page per commercial value, so any number can be checked against its document in seconds.

The prepared example ships as references/exercise-data/Quote_Comparison_Workbook.xlsx (sheet "Raw Extraction"). Grade your run against the documents first, Appendix B.3 second.

Step 8: compare the quotes - and refuse to pick a winner

Comparison is where invented certainty creeps in, so the prompt makes the rule explicit: normalize only what has a stated basis, show the formulas, and keep the original values visible next to every derived one.

Using the Raw Extraction sheet, create a Normalized Comparison sheet.

Normalize quantities, units, or currencies only when the conversion rule or basis is supplied. Keep every original extracted value visible. Flag differences in scope, assumptions, exclusions, and commercial terms. Show formulas for comparable totals.

Do not identify a best quote until the comparison basis is complete. List the missing information and unresolved questions that prevent a fair recommendation.

The comparison prompt, verbatim - workbook tab Quote-Compare.

Run on the three sample documents, the arithmetic itself is easy. At the quoted 5,000-unit volume, with one-time tooling spread across the units: Alpha Components lands at USD 15.60 per unit (13.90 plus 8,500 of tooling over 5,000 units), FOB - the buyer arranges freight from Alpha's dock. Bravo Plastics lands at EUR 14.70 per unit (11.90 plus a 2.80 tooling share), EXW - the buyer collects at the factory, freight and duties excluded. Cardinal Metals is USD 16.40 all-in, DDP - delivered with freight and duties paid, tooling waived. And that is exactly where a careful comparison stops: Bravo is quoted in euros with no exchange-rate basis supplied, and the three delivery terms put freight, duties, and risk on different sides of the line.

CHECK — The correct answer is "not yet"

The sample quotes are engineered so the honest deliverable is a comparison that stops - naming the euro conversion basis, freight, and taxes as blocking gaps - rather than a winner. If your run picked a best quote, it invented at least one basis; find which.

The prepared sheet ("Normalized Comparison" in the same workbook) and Appendix B.3 show the full expected shape.

Step 9: reference files become a research workbook

The last live step generalizes everything the quotes taught. The material is a research pack - references/exercise-data/research-pack/, four fictional files: a quality-manual excerpt, a supplier-quality newsletter, an undated scorecard guideline, and an 8D problem-solving one-pager - and a research question: how should supplier quality reviews be run? The deliverable is a workbook in three sheets: Evidence, Synthesis, Sources.

Review the attached reference files and build a research workbook using only information you can trace to a source.

Create an Evidence sheet with one claim, instruction, or finding per row and these columns: ID, topic, claim or instruction, source file, author or organization, date, page or section, short supporting excerpt, plain-language interpretation, caveat or limitation, relevance to [research question], confidence or status, and open question.

Create a Synthesis sheet that orders the findings from foundations to implications and identifies agreements, conflicts, and gaps.

Create a Sources sheet that indexes every file.

Write 'Not stated' when information is missing. Keep conflicting claims separate. Do not invent citations. Add filters, freeze the header row, and wrap long text. Deliver an editable Excel workbook. If the tool cannot create an Excel file, provide separate comma-separated values (CSV) tables.

The research prompt, verbatim - workbook tab Research.

The pack is engineered the way real source material behaves: two of the sources disagree on review cadence - the manual excerpt says quarterly, the newsletter says monthly - and the scorecard guideline carries no date at all.

CHECK — Conflicts stay visible

The Evidence sheet must carry both cadence claims as separate rows, the Synthesis sheet must name the conflict as a conflict, and the undated guideline keeps "Not stated" in its date column. A tidy synthesis that quietly picks a side is a defect - however confident it reads.

The prepared example ships as references/exercise-data/Research_Workbook.xlsx - twelve traced evidence rows, a synthesis that names the agreements, the conflict, and the gaps, and a source index. Self-check: Appendix B.4.

Reference exercise: explain a topic clearly

One artifact steps out of the live hour into the reference layer: the plain-language document. Run it as self-study with the same requirement discipline. It corrects a common misunderstanding: explaining something simply is a requirements problem - audience, vocabulary, fidelity, and a learning check - not dumbing down. The dominant risk is not that the text stays too hard; it is that the simplification quietly changes the meaning: a dropped condition, an analogy stretched past where it holds, a caveat that vanished.

The requirement types: **reader and purpose, vocabulary, structure, examples and analogies, fidelity and caveats, tone**, and the **comprehension check**. Reading level and fidelity dominate. The plain-language table in references/Requirements_by_Artifact.md carries the full menu with acceptance evidence per type.

Using only the supplied source, explain [topic] at about a fifth-grade reading level. Start with the main idea. Use short sentences, define every acronym, and include one accurate everyday example. Preserve important limits and conditions. End with three comprehension questions and a separate answer key. Then list any important detail the simpler version leaves out.

The plain-language prompt, verbatim - workbook tab 5-Explain-Clearly (reference exercise).

Every clause is a requirement you can check. Main idea first, so a reader who stops after one paragraph still leaves with the point. Analogies only where accurate, with their limit stated. Conditions and uncertainty preserved. A respectful adult tone: fifth-grade reading level describes sentence difficulty, never the reader's intelligence. And two closing checks: three comprehension questions answerable from the text alone, and a self-audit against the source for any simplification that changed the meaning.

The worked topic: reorder points and safety stock

The demonstration source is the SOURCE section of references/exercise-data/plain-language/inventory_replenishment.md: reorder points, safety stock, holding costs, and one load-bearing caveat - the formulas assume the past predicts future demand, so parameters must be reviewed periodically. The sample output (printed in Appendix A.5) puts the main idea in the first sentence. It uses a bakery for the example and a car's fuel light for the analogy, and it states the analogy's limit: a fuel light is set once, a reorder point must be reviewed. The caveat survived the simplification; that survival is the exercise.

CHECK — Three success checks

Main idea in the first sentence; the analogy's limit stated; the review caveat survived the simplification. If any of the three is missing, the output failed - however pleasant it reads.

Finding the simplifications that changed meaning is part of the exercise: compare the explanation with the source sentence by sentence - a missing condition is a meaning change, not a style choice. Each sample file in the pack ships with its own fidelity note, because the note is part of the deliverable - a plain-language document that cannot say what it dropped has not finished the job.

MYTH — A readability score proves understanding

It does not. Reading-level estimates such as Flesch-Kincaid are supporting evidence, one signal among several. The proof is a human check - which is why the prompt ends with three comprehension questions answerable from the text alone.

Three variations, same requirements

The pack holds three self-study variations beside the worked topic, each with a SOURCE text and a sample output: household budgeting (household_budgeting.md), the water cycle (water_cycle.md), and how an internet message travels (internet_message.md), all in `references/exercise-data/plain-language/` with the checks in the pack README.

TRY IT — Run a variation blind

Run the prompt on the SOURCE section of `water_cycle.md` or `internet_message.md` and grade your output against the checks before opening the sample. Then hunt your own output for one simplification that changed meaning - there is usually at least one.

The arc, complete

Nine steps, one method. Every prompt in this chapter had the same skeleton: sources and scope, selected artifact requirements, acceptance evidence, boundaries. And every step ended with a check you could actually perform - a reconciliation, a filtered result, a coverage list, four planted traps, a comparison that refuses to name a winner, a conflict kept visible.

Artifact	Requirement types that dominate
Spreadsheet analysis	Calculation rules: row grain, period, metric definitions, denominators, missing-data treatment
Returned Excel workbook	File contract: returned-file naming, a Summary that reconciles, native editable charts, traceable formulas
Interactive dashboard	Behavior and state: controls, coordinated views, empty vs. missing vs. zero, offline delivery
Management presentation	Narrative and editability: audience and decision, message hierarchy, data fidelity, speaker notes
Email brief	Chronology and evidence: current state, decisions with conditions, ownership, authority boundary
Quotation extraction	Fidelity: verbatim values, original currencies and units, "Not stated" for gaps, a citation per value
Quotation comparison	Basis discipline: normalize only on a stated basis, visible formulas, no winner while gaps remain
Research workbook	Traceability: claim-level sourcing, conflicts kept visible, confidence and open questions per row

Artifact	Requirement types that dominate
Plain-language document	Reading level and fidelity: vocabulary, analogy limits, preserved caveats, comprehension check

KEY — The connecting insight

A requirement is only as good as the acceptance evidence attached to it. For every type you select, write: requirement, example, evidence.

Everything in this chapter is packaged for reuse: the prompts sit in copy-paste tabs of deliverables/Course_Workbook.xlsx in training order, the per-artifact menus in references/Requirements_by_Artifact.md, and deliverables/Prompt_Template_Creator.html assembles prompts from the same families (Chapter 6 shows how to save one). When your task is none of the nine, the skeleton still holds.

One thing was constant across all nine steps: you asked for output and inspected it yourself. The next part changes that relationship - handing over a job the assistant runs, checks, and must know when to stop and ask you about. That is the move from asking to delegating.

PART 5 - AGENTIC WORK

Agentic work: delegating with boundaries

Everything so far has asked the model for text you would read, judge, and use yourself. This part crosses the line the course is named for: from asking for an output to delegating a job. That changes what a prompt must contain — environment, process, checks, and the moments the work must stop and come back to you — because actions, unlike text, change the world.

What makes a workflow agentic

An agent is a model given tools and instructions, run in a loop. Four ingredients must be present. A **goal**: a defined outcome with a completion condition, so the loop has an exit. **Tools**: capabilities to read, calculate, create, or act. An **action loop**: the result of each step informs the next. And **control**: permissions, checks, and human review at the required boundaries.

The loop: gather context, act, verify against the goal, adjust or stop — until the goal is met, a check fails, or a guardrail says ask the human. Handed the supplier task, an agent would inspect the workbook, calculate, check its totals, and produce the scorecard — choosing its steps within its authority, while you define the goal and review what matters. A chatbot that answers and waits is not an agent; a long answer or a clarifying question alone does not establish agency.

One demystifying detail: agent memory is files. The context window is short-term memory; notes and logs are long-term, and nothing survives unless written down. An agent summarizing its thread into a note and reloading it is the foundations chapter's handoff move, automated.

ASK and DELEGATE: the decision, not the tools

The central distinction of this part is not a vendor label; it is a decision about work and authority. ASK and DELEGATE were felt in Chapter 1. What delegation adds: the agent acts, so the brief must say in advance what may happen without another instruction and where review is required.

	ASK (generative)	DELEGATE (agentic)
Specifies	what to write	a job to run
Contains	role, task, context, format, examples	all of that, plus mission and done, boundaries, gates, checks, logs
Produces	one response you read	actions, artifacts, a report with evidence
Failure mode	misleading text — re-prompt	wrong actions: files changed, emails sent
Your role	reader and editor	delegator, reviewer at checkpoints
Think of it as	briefing a ghost-writer	a work order for a contractor

The agentic column is a superset: every generative element still applies, wrapped in a job. Both modes require judgment - a chat response can still cause harm if used unchecked - but delegation adds responsibility for the actions taken, and that difference in failure cost drives everything this

chapter adds.

KEY — The heart of the course

ASK requests an answer; DELEGATE commissions bounded work with evidence and checkpoints. Remember one contrast from the course: this one.

Which way of working fits: a self-test

Before writing any brief, pick the vehicle. In **chat**, you are the loop: exploring, drafting, judging, steering every turn. In a **workflow**, the loop is frozen: known path, few tools, the same steps every time — script it in an automation platform; cheaper, predictable, auditable. In an **agent**, the loop runs inside: the next step depends on what the work finds, so you brief it, with gates where the judgment stays yours. Do not chat with an agent — hand it a work order; your control lives in the brief, not the conversation.

The decision rule: known path and few tools — workflow. Unknown path and real judgment — agent, with gates. Still thinking it through — chat. The wrong vehicle costs both ways: an agent on a known path pays judgment prices for clerk work; a workflow on an unknown path breaks on the first surprise.

Three scenarios as a self-test

- Explore three possible ways to explain a supplier problem.
- Apply the same approved calculation to a monthly export.
- Investigate inconsistent files, choose the next check, and prepare a draft report.

Typical answers run chat, workflow, agent — but the assumptions matter more than the label: exploration needs your judgment every turn; an approved calculation needs none mid-run; an investigation cannot be scripted. Risk and permission constraints can still change the implementation even when the task is a good technical fit.

TRY IT — Name the control you keep

For each scenario, name the one approval boundary you would not give away. If you cannot, you are not ready to delegate.

The anatomy, grown up to survive autonomy

The agentic brief is not a fresh invention. Every block is a generative element from the craft chapter, grown up to survive autonomy — plus conduct blocks that text-only prompts never needed. This mapping is why the twelve-block brief below is inevitable, not arbitrary.

What changes	Why
Every element must survive unsupervised decisions: role as behaviors only, context as a per-source register with grain and trust, examples as a worked run	the agent decides for hours without you; a vibe or a guess propagates through the run

What changes	Why
The task gains a definition of done; the Stop becomes gates and autonomy rules	the loop needs an exit condition, and scope control becomes checkpointed process
The format gains logs and a fixed report shape; the Out becomes a logged, owned open item	actions must be traceable, and not-knowing must be owned
Conduct blocks appear with no generative ancestor: environment, plan, checks, rules, quality bar	method, verification, and invariants that a text-only prompt never needed

The mission brief for bounded work

A usable mission brief answers four questions. **Outcome:** what finished work should exist, for whom, by when? **Inputs and scope:** what may the agent read, use, change, and leave untouched?

Process and authority: what may proceed on its own, and what requires human review? **Checks and delivery:** what must pass, what stops the run, what evidence is delivered? Behind them sits a twelve-block design checklist — a reference aid, not twelve mandatory paragraphs; each block exists because a specific failure taught someone to add it.

- `<role>` — behaviors, not titles: "never invents numbers" beats "senior analyst".
- `<mission>` — one goal, audience, deadline, plus the line chat never needs: "done looks like...".
- `<context>` — glossary, quirks, targets: tribal knowledge the agent cannot infer.
- `<environment>` — where to read and write, what is forbidden; inputs read-only, outputs versioned.
- `<inputs>` — per source: location, format, what one row means, trust level; the unestablished is UNKNOWN, never guessed.
- `<plan>` — steps in order; methods allowed; methods needing permission first.
- `<checks>` — HARD stops the run, SOFT flags; evaluable equalities, with one reconciliation anchor you trust.
- `<outputs>` — exact deliverable names, plus three logs: changes, findings, open items.
- `<process>` — phases with human gates at cheap-error points; autonomy rules for proceed versus stop-and-ask.
- `<rules>` — non-negotiables: never invent data; sources immutable; every transformation logged.
- `<quality_bar>` — done as an auditable checklist; prevents "done" meaning "I stopped".
- `<reporting>` — the fixed final message: status, headline with the number, evidence, open items.

Two disciplines keep this honest. The **operating-manual test:** if a colleague could not run the job from the brief without asking a question, the agent will guess — fix the brief, not the run. And **design thick, ship lean:** think against all twelve blocks, but ship the one-screen plan-do-check-act brief from the craft chapter — conflicting rules destabilize more than missing detail. Block-by-block depth: references/Elements_of_Prompting_Field_Guide.pdf.

Worked exercise: the supplier analysis becomes a mission brief

You already know the ASK version of this job: the fully specified analysis prompt from Chapter 3, "One task through the improvement loop" - answered while you watch.

Now delegate it: prepare a supplier scorecard from the supplied workbook. Boundaries: keep the source unchanged; confirm metric definitions before calculating; reconcile current-source totals. Delivery: the scorecard, assumptions, failed checks, open questions — and do not send it to anyone. Reusing the supplier case is deliberate: the concept being practiced is delegation, not a fresh business problem.

Shaped as a mission, the work becomes five phases. **Extract**: read-only; profile every input; fix nothing yet. **Transform**: one schema (agreed column layout), ordered rules, each logged with rows affected. **Validate**: reconcile to a known total; a HARD failure stops the run. **Analyze**: answer the numbered questions, then stop — "associated with", never "caused by". **Present**: headline titles carrying the number, sample size on every proportion, a source-plus-as-of footer. Each phase ends by emitting a checkable claim — a total, a row count, an assumption — and the next begins by checking it.

Three human gates sit where errors are still cheap. **Gate 1 - definitions**, before any data is touched: the agent restates metrics, schema, and assumptions, and you confirm. A wrong denominator dies here for pennies. **Gate 2 - reconciliation**: computed totals must tie to a number you already trust. Here that is the workbook's TOTAL row - the shipped and returned totals that Chapter 4, step 1 reconciled to - with the one blank Inspection_Hours cell disclosed as missing, not zeroed. **Gate 3 - headlines**, before rendering: titles and bullets as plain text. Re-rendering is cheap; re-thinking a circulated deck is not. The correct headline is step 1's finding - Bravo Plastics has the highest return rate - with its number in the title.

Between gates the agent proceeds within its stated permissions, asking only if a HARD check fails, an input is not as described, a definition is ambiguous, or a previously published number would change. Without "proceed" you built a slow chatbot; without "stop", a liability.

TRAP — The forced match

A failed reconciliation must produce a discrepancy report — offending rows, both totals, the gap — never a forced match.

TRY IT — Your turn

Write the brief in four labeled lines — source, authority, check, deliverable — then add one pause condition and one piece of completion evidence.

Choosing an environment for delegated work

Product names change quickly, so choose where delegated work runs by category, and verify the current implementation in the tool you actually have.

- A coding environment — when the work involves source files, scripts, builds, or reproducible calculations.
- A workplace assistant — when approved documents and office workflows are central to the task.
- An automation platform — for a defined sequence with stable triggers and repeatable steps.
- An agent environment — when intermediate findings determine the next action.

Whatever the category, compare four things before committing a live workflow: capabilities, permissions, review points, and the evidence a run leaves behind.

Instructions versus enforceable controls

Your authority over an agent lives in two layers. **In the brief:** define allowed actions, name the approval points, require evidence of completion. **In the environment:** limit file and tool access, configure which actions need approval, keep a recoverable version of the work. The brief requests behavior; the environment restricts what is possible. A gate written in the prompt is a suggestion; a gate configured in the tool's permission settings is a control - Singapore's regulator, 2026: prompt guardrails "are not fail-safe".

Five rules for delegating

- Reversible? Proceed. Irreversible? Ask. Scratch edits are reversible; deleting, sending, publishing, overwriting a master is not — the brief must say stop and ask first. This is the eagerness dial.
- Least privilege: minimum access, shortest time. The UK's National Cyber Security Centre: "If you cannot understand, monitor or contain an agent's actions, it is not ready for deployment."
- Prompt injection is real: anything the agent reads can carry hostile instructions. Danger peaks when private data, untrusted content, and an outbound channel combine; break any one leg.
- Vet third-party skills: executable code plus instructions. The top-ranked community skill for one popular open agent was silently exfiltrating data (Cisco, January 2026). Inspect before install.
- Demand evidence, not assertions: "done" means the check it ran, the output it got, the file re-opened and verified. Latitude grows with track record.

The injection rule reframes how an agent must read: text inside a source document is evidence to inspect, never authority to change the task - a cell saying "ignore your instructions" is quoting, not commanding. Write that framing into briefs as defense in depth. The course's concrete boundary is a good default: work on a copy of the workbook and a draft report; nothing sent, no master overwritten.

Diagnosing a failed run

The craft chapter's grid (top of the workbook's EX-Report tab) diagnosed the answer: name the failed element, repair it. This is its agentic twin: when the run is wrong, name the context failure, apply its cure, rerun the affected step, and verify the result.

Failure	The smell	The cure
Poisoning	one bad "fact" entered early; later steps politely build on it	fresh session; reload only the standing brief and verified files
Drift	a long run wanders; the middle of a huge context gets skimmed	re-inject the mission; compact (the handoff move); stop conditions in the brief
Confusion	too many tools on the desk; it picks the wrong one or dithers	fewer tools per phase; the plan names the methods allowed
Clash	two sources disagree; the run silently picks one	one source of truth per fact, named in the inputs register

The common frustration — "why did it ignore me?" — usually has a structural answer. Instructions stack in layers: the vendor's system prompt, your standing-instruction file, any skill the task loads,

and today's message — the easiest layer to lose. Precedence varies by product, so find which layer said what before blaming the model. One-job rules belong in a skill; and tool descriptions are prompts too — a badly named tool is a badly written prompt.

Write the brief once — the workspace remembers

Supervision costs effort; here is where agentic prompting pays it back. The first habit is a **standing-instruction file** — read by the agent at the start of every session, the onboarding packet for your workspace. Include folder layout and read/write boundaries, conventions, standing rules and the ask-first list, known data quirks; exclude anything inferable from the files. Keep it to roughly sixty lines, audited line by line: would removing this cause mistakes? If not, cut — bloat drowns your real instructions. Never include credentials or personal data: the file is read every session and lives in history.

The second habit is **reuse by diff**. The full mission brief is written once and stored in the workspace; each cycle's prompt is then about ten lines: input paths, the as-of date, deltas to watch, any extra gate. Report by delta too: compare every headline metric to the last cycle, with one-line explanations for moves beyond a threshold. When a definition changes at a gate, update the standing brief in the same session and log it. After two "just this once" exceptions, fold them into the brief.

KEY — The economics

Briefs stop being prompts and start being investments: the thinking amortizes over every future cycle.

Exercise: one gate and one hard check

Close the part with the smallest useful act of delegation design. Pick one job you would actually delegate - a recurring report, a folder to process - and write two lines. A **gate** controls an action: "Pause before ___ until ___ is confirmed." A **hard check** tests a result: "Stop and report the discrepancy if ___." Make the check an equality the agent can evaluate, anchored to a number you already trust. "Check data quality" is an aspiration, not a check.

The course's example: detail totals must reconcile to the current source under the same filters and units - the Berlin check from the dashboard step (8,575 units, 21 returns, 0.245%), decidable because filters, units, and period match on both sides. One caution: a fresh reporting period does not need to equal the old one. Reconcile like against like, and investigate period-to-period differences separately.

TRY IT — Write the two lines now

One gate: what must be confirmed before the agent touches anything? One hard check: which equality, anchored to which trusted number?

CHECK — The debrief question

Where the gate felt hard to write, the process was fuzzy all along — a finding about the process, not the agent.

The final part turns from doing the work to keeping it: reusable prompts as assets your team maintains. The compact ASK-versus-DELEGATE summary is on the cheat sheet.

PART 6 - REUSABLE PROMPTS

Reusable prompts: from one good result to a shared asset

A prompt that survives the improvement loop is worth more than the answer it produced. This final part turns the prompts you refined during the course into named, tested, owned assets — and sends you off with one saved for Monday, the habits that outlast any tool, and the capstone that proves you can apply them.

A useful prompt becomes a reusable asset

A good prompt took an hour of iteration to earn. Re-typing it from memory throws that hour away — and gives your teammate a worse version than yours. That is the whole argument for prompt management. The prompts you built against the supplier workbook and the packaging thread are small work specifications that have already passed a check; treat them like any other controlled document.

Picture the same prompt promoted: sticky note, then binder, then library shelf. The promotion ladder names five rungs, and each climb is earned by repeated use, not enthusiasm.

Rung	What it looks like	When it is the right home
1. One-off prompt	Typed, used, gone	Genuinely one-time asks
2. Personal library	A saved-prompts doc or text-expander snippets	Zero infrastructure; start today
3. Team library	A shared database with one tested template	When your prompt becomes everyone's baseline
4. Packaged and auto-applied	Project instructions, skills, packaged agents	When the prompt should load itself as context matches
5. Prompts as code	Standing-instruction files in a repo, reviewed like code	Where agentic briefs live

Rung 4 is where consistency stops depending on people remembering. Most people belong at rungs 2 and 3 within a month.

KEY — The promotion trigger
Anthropic's rule of thumb: explained the same task more than once? Package it — together with its reference material.

Two cautions. Do not package an unreliable prompt just because it is used often. And use a simple shared document when it meets the need - many teams get most of the value from a well-kept rung 3. Whatever the storage method, the source assumptions and checks travel with the prompt. If your site runs controlled work instructions - name, owner, revision, review date - run the library the same way: a process analogy, not regulatory equivalence.

Where reusable prompts can live

Choose the simplest location that supports the intended users and the review process. Wherever you already work, there is a home for reusable prompts. The map below is as of August 2026; features

move, so re-verify before you standardize.

Tool	Home for reusable prompts (as of August 2026)
ChatGPT	Projects (files plus instructions); custom GPTs, transitioning to workspace agents on business plans
Claude	Projects (instructions plus knowledge); Styles for tone; Skills — a folder with SKILL.md that loads itself when relevant
Gemini	Gems (instructions plus knowledge files, shareable via Drive permissions); saved prompt "skills" in Chrome
Copilot	Prompt Gallery — save, share to a team, admin-auditable, inside your tenant; Agent Builder for packaged agents
Any tool	A team library with fields: name, purpose, template, owner, version and change note, models tested, last tested, example output, status
Agentic (repo)	CLAUDE.md or AGENTS.md at the root; briefs and skills in versioned folders, reviewed like code

No such products? The "any tool" row is the takeaway: a plain shared document with those fields is a working team library.

TRAP — A naming collision

"Skills" is a packaged folder at Anthropic but a saved prompt in Chrome's Gemini — check which one a colleague means.

KEY — Tools change; the practice does not

Prompt-tooling startups get acquired and shut down yearly — Humanloop, an early leader, shut down in September 2025 when Anthropic hired its team. Version it, test it, own it: the discipline outlives every platform.

A small review process for shared prompts

The moment a prompt is shared, other people inherit its failures. Before you mark one approved, define success first, then run it against three representative cases:

- Typical case - does it produce the expected useful result?
- Edge case - does it handle a missing value or a changed condition?
- Unanswerable case - does it expose the gap instead of inventing an answer? This is the case teams skip, and it tests whether the prompt's out survives pressure.

A replacement must solve the intended problem without introducing critical regressions - it has to beat the version it replaces on those cases, not just feel better. Re-run the cases after every model upgrade.

A few conventions keep a library alive:

- {{placeholders}} for everything that changes per run - search for "{{" before running.
- [optional] blocks you delete when unused.
- A version and a one-line change note naming the model tested.
- A filled example next to every blank template - the gold standard of what good looks like.
- A named owner and review cadence for every approved prompt.

TRAP — Prompts contain data

A stored prompt often carries supplier names, pricing, findings, or draft language — classify the library like the documents it quotes. Never store credentials or personal data in a template; placeholders supply them at run time. Shared means distributed: owner approval first, and audit third-party packaged prompts like software.

Save one prompt for Monday

This is the exercise the whole course cashes out into, and it takes two minutes. Do not start another task: take a prompt you have already improved — supplier analysis, dashboard specification, email summary — and save it properly.

- Choose one prompt you will use again.
- Save it with a useful name, a filled example, its source assumptions, and a check.
- Add a version and an owner if you plan to share it.

Pick a realistic storage location, not a software system you would have to introduce first — a saved-prompts document counts. Note the single source assumption most likely to change; that is the line you will edit first when the data or the tool moves.

TRY IT — Your first library entry

Answer three questions in writing. Which rung of the ladder are you on today? What is the one prompt you already reuse from memory? Where will it live, and what is its first version line? Answer them and your prompt library exists.

CHECK — Success check

Another person can tell, from the saved entry alone, when to use the prompt and how to check its output. If they would have to ask you, the entry is not finished.

The package gives you two saving surfaces. **Prompt_Template_Creator.html** (in deliverables/) is the builder: pick a task family, tick the requirements that resolve ambiguity, edit any wording, and copy the assembled prompt — a counter tracks unfilled {{placeholders}}, work saves in your browser, and nothing you type leaves the file. **Course_Workbook.xlsx** carries the course prompts in tabs 1-Analyze-Data through Research, in training order (with 5-Explain-Clearly as the reference exercise), each with a filled example, commentary, and its check. The email prompt, in the fuller form a library entry deserves:

Review the selected conversation about [topic] using only the messages and attachments you can access. Create a brief of the current situation, followed by tables of decisions and actions. For each decision: status, conditions, supporting message. For each action: task, explicitly stated owner, due date, status, supporting message. Distinguish current dates from superseded dates. List unanswered questions, conflicts, missing information, and referenced attachments you cannot access. Preserve approval conditions. Write "Not stated" for missing details. Separate source facts from your interpretation. End with the next clarification needed. If requested, draft a reply for review; do not send it.

The fuller email-brief template; Reference Deck slide 103 carries a condensed version of it. The live exercise uses the shorter structured brief (Chapter 4, step 6); this is the library-entry form, with [topic] filled at run time.

Read it as a library entry and the conventions are visible: a placeholder for what changes per run, an out ("Not stated") with a checkable shape, and an authority boundary (draft, do not send). Its check

is stored beside it: Appendix B.2 lists the traps a good summary must catch.

Let the assistant interview you

One skill remains, and it points the other way. All course long you wrote requirements for the model; the last move is letting the model surface the ones you missed. Ask a clarifying question when a material unknown would change the task, the output, or the acceptance check; otherwise state a reasonable assumption and produce a reviewable draft. Make questions specific — "Which decision should this report support?" beats "Tell me more" — and notice that a missing requirement type often tells you which question to ask.

Before answering, ask me clarifying questions until you're 95% confident you understand what I need.

The flip — one line you can paste at the top of any consequential request.

WORKS — Why the flip works

When the model asks first, accuracy on unclear requests improves by roughly 7 to 14 points (ClarifyGPT, FSE 2024). Answering its questions is quality control, not overhead.

The question toolkit works on you as well as on the model. **Five Whys** — Taiichi Ohno's Toyota discipline — digs from symptom to cause. The **open-to-closed funnel** explores first ("what is driving this?") and verifies second ("so the cap is 18,000 — yes or no?"). The **Socratic check** interrogates any AI answer: what is assumed, what is the evidence, what if we are wrong?

KEY — The two skills are one

The element you cannot fill in yet is the exact question to ask - or to let the AI ask you. Sam Altman, 2025: figuring out what questions to ask will matter more than the answers. Picasso, 1964: computers "can only give you answers."

The habits to carry into work

Three habits compress the course. **Specify**: choose the requirement types your artifact needs — content, behavior, format, checks. **Verify**: check the result against evidence appropriate to the task. **Delegate**: define the work, authority, completion checks, and review points. And hold your library to one bar: a saved prompt is useful when you can explain why it works and where it can fail.

Ten things worth remembering

- The prompt is the whole steering wheel — everything the model knows about your task must be in it, or in files it can read.
- Anatomy beats inspiration: Role, Task, Context, Format, Examples — plus the Out and the Stop.
- The context window is a desk, not a filing cabinet: documents at the top, question at the end, fresh chat per topic.
- Numbers come from code execution, quotes come from documents, current facts come from search — never from free recall.
- Fluency is not evidence. Uncited claims are drafts. Give the model an out and demand citations.
- Never reveal your preferred answer when asking for judgment — AI affirms you about 49% more than a human would.

- On thinking models, drop the step-by-step scripts and keep the clarity; contradictions cost real money.
- A generative prompt says what to write; an agentic prompt is a work order: mission, environment, checks, gates, autonomy.
- Gates catch errors cheapest: definitions before data, reconciliation before analysis, headlines before rendering.
- Prompts that work are assets: name them, version them, store them where the team — and the agent — can find them.

KEY — One sentence

If you keep only one sentence: a prompt deletes wrong guesses, binds the job, and decides what sits on the desk.

The capstone: one task, one checked result

The exit ticket is a single completed rep. **Choose** a real prompt from your own work, or fall back on the supplier or email example. **Improve and run**: clarify the one missing requirement, then generate. **Check and explain**: show the evidence your check produced and name one revision you would make. Three routes qualify, and one is enough: repair a prompt, review a document against a rubric, or write a mission brief. A full agent run is not required - writing a sound brief is the skill. If generation is slow, evaluate a prepared result against the same criteria; the judgment is the rep, not the waiting.

CHECK — Capstone success

A clear task, grounded output, an explicit check, and a reusable next version. All four on one small task beats three of them on a large one.

Make it stick: three fifteen-minute follow-ups

- Anatomy rebuild. Rebuild a prompt you actually used last week with the seven-element anatomy; run both and compare. Then ask the model to improve the rebuilt prompt and run that too - you will see the quality jump, and how cheap it was.
- Blind review. Rerun the Chapter 1 mirror test on a document you own - without hinting at your view.
- First mission brief. Pick a small recurring job with files and write the full brief, placeholders and all. No agent needed - writing a sound brief is the skill.

Run them on work you actually own — toy exercises do not transfer. Then set a share-back: at the next team meeting, two volunteers show a before and after. Public commitment turns a training into a practice.

Your next workplace application

One task. A clear brief. A checked result. That is the whole assignment. Pick the task this week, choose the requirement types its artifact needs — references/Requirements_by_Artifact.md carries the menu per artifact — write the brief, run it, and check the result against its own acceptance evidence. Decide with judgment; delegate within clear boundaries.

The course ends where your work begins. **Draft with AI. Decide with judgment. Delegate with a brief.** And the last skill of all — when you are not sure what to ask for, let it ask you. Save one prompt tonight, run one checked task this week, and the practice will build itself the way every practice does: one named, tested, owned improvement at a time.

APPENDIX

Reference: the glossary, the criteria, and the evidence

This appendix is the layer of the course you return to after the reading is done: the vocabulary, the evidence behind every requirement type, and the map of the package itself. Use the glossary while you read, the criteria maps when you wonder whether a slot is worth filling, and the evidence verdicts when a piece of prompting advice needs one.

The glossary: twenty terms that matter

Every term below was used in context in this course. Two entries change behavior rather than vocabulary: **sycophancy**, the reason you blind a review, and **gate**, the checkpoint that makes agents safe to delegate to.

Term	Meaning
Escalation ladder	Prompt first, retrieve second, fine-tune last; the capability triage of Part 1.
Promotion ladder	One-off -> personal -> team -> packaged -> as-code; where a prompt lives (Part 6).
Token	The text chunk a model actually reads (about three-quarters of a word); pricing and limits count these.
Context window	Information available for a response; distinct from saved chat history.
Knowledge cutoff	Where training data stops; anything after needs search or your documents.
RAG	Retrieval-augmented generation: fetch relevant passages, answer from them, with citations.
Embedding	A text's coordinate in meaning-space; powers semantic search and RAG retrieval.
Hallucination	Fluent, confident, wrong; plausibility optimized instead of truth.
Reasoning model	Uses additional computation before answering; assess benefit and cost on your task.
Few-shot	Providing a few examples in the prompt.

Term	Meaning
Chain-of-thought	A generated sequence of intermediate steps; it still needs verification.
Metaprompting	Asking the model to critique or rewrite your prompt.
Sycophancy	The trained tendency to agree with you; blind the review to defuse it.
Agent	Model + tools + instructions in a loop, acting until done or stopped.
MCP	Model Context Protocol: the open standard for plugging tools into agents.
Mission brief	An agentic prompt: goal, environment, inputs, process, checks, gates, reporting.
Gate	A human checkpoint inside an agent's process; approve before it proceeds.

Term	Meaning
Open weights	Downloadable model weights; privacy depends on the deployment.
Taxonomy	The element -> attribute -> option catalog behind the Template Creator; your parts list.
Context engineering	Choosing everything on the desk: prompt, files, history, tool results; parent of agentic prompting.

Why each dial matters: the generative criteria

This map lists each generative requirement type with the source behind it. What filling the slot does to your output is explained where the element is introduced in Chapter 3; the field guide carries the full rationale.

- **Identity and role** (Zheng 2024; Salewski 2023)
- **Behavioral rules** (Anthropic hallucination docs)
- **Stance and blinding** (Cheng, Science 2026; Sharma 2023)
- **Task verb and question** (Google guide; OpenAI GPT-5 guide)
- **Purpose, the why** (Anthropic best practices)
- **Context and glossary** (OpenAI 2025)
- **Material placement** (Anthropic long-context; Liu 2023)
- **Format and length** (Sclar, ICLR 2024)
- **Examples** (Brown 2020; Lu 2021; DeepSeek-R1)
- **The out and the stop** (Anthropic docs; OpenAI 2025)

CHECK — How to read evidence claims

Every claim travels with its task, model, date, and limitation. Check the original publication when a specific claim matters to your decision.

Why each dial matters: the agentic criteria

The second map lists the agentic criteria, each with the mission-brief block it governs and the source behind it; the rationale per block is in Chapter 5, "The mission brief for bounded work". Use it whenever a task introduces new tools, actions, or approval boundaries.

- **Posture and standing rules** - <role>, <rules> (Anthropic harness guidance)
- **Done plus verifiable-by** - <mission> (Anthropic memory-tool guidance)
- **Environment boundaries** - <environment> (Claude Code protected paths)
- **Least-privilege tools** - <environment> (OpenAI practical guide; NCSC)
- **Input register and trust** - <inputs> (pipeline and register discipline)
- **Plan steps and methods** - <plan> (Anthropic chaining; Faith and Fate 2023)
- **Checks: anchor, hard, soft** - <checks> (Anthropic Agent SDK)
- **Gates and autonomy** - <process> (OpenAI guide; Feng et al. 2025)
- **Logs and reproducibility** - <outputs> (Anthropic harness guidance)
- **Report shape and discipline** - <reporting>, <quality_bar> (SDK status enums; harness docs)

The evidence verdicts: works, myth, expired

The course sorts techniques into three verdicts: **works**, **myth**, and **expired**. The lists, with each verdict defined, are in Chapter 3, "Prompt techniques to test"; the full compendium, with the study behind every line, is in references/Elements_of_Prompting_Field_Guide.pdf and the PLAYBOOK tab of Course_Workbook.xlsx, researched as of September 2026.

Two honest edges. Politeness engineering is inconsistent in both directions: be normally civil, do not engineer tone. And "prompt engineering is dead" is a half-myth: the job title faded, but phrasing still swings results by double digits on frontier models.

Bibliography

Every evidence claim in this course traces to a study, a vendor document, a standard, or a dated snapshot. This bibliography expands the deck's sources page into full entries, grouped by kind. Volatile items (leaderboards, product claims) carry their retrieval date and are refreshed quarterly; the dated research record behind each entry ships with the package source (notes/research/, files r1-r28 and t1-t6, each with retrieval dates).

Research papers and studies

- Brown, T., et al. (2020). "Language Models are Few-Shot Learners." NeurIPS 2020. arXiv:2005.14165. — few-shot prompting.
- Wei, J., et al. (2022). "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." NeurIPS 2022. arXiv:2201.11903.
- Kojima, T., et al. (2022). Zero-shot chain-of-thought ("Let's think step by step"). NeurIPS 2022. arXiv:2205.11916.
- Wang, X., et al. (2023). Self-consistency over sampled reasoning paths. ICLR 2023. arXiv:2203.11171.
- Liu, N. F., et al. (2024). "Lost in the Middle" — long-context position effects. TACL. arXiv:2307.03172.
- Sharma, M., et al. (2023). "Towards Understanding Sycophancy in Language Models." Anthropic. arXiv:2310.13548.
- Sclar, M., et al. (2024). Prompt-format sensitivity — up to 76 accuracy points from trivial formatting changes. ICLR 2024. arXiv:2310.11324.
- Zheng, M., et al. (2024). "When 'A Helpful Assistant' Is Not Really Helpful." EMNLP 2024 Findings. arXiv:2311.10054. — personas and accuracy.
- PersonaLLM (2024). NAACL 2024 Findings (ACL Anthology 2024.findings-naacl.229). — personas shape voice, not correctness.
- Salinas, A., and Morstatter, F. (2024). Prompt perturbations — "even the smallest of perturbations ... can cause the LLM to change its answer." arXiv:2401.03729.
- Wharton Prompting Science Reports 1-4 (2025; Report 4: Dec 7, 2025, SSRN 5879722). — politeness, tips/threats, and the six-model persona replication.
- Sprague, Z., et al. (2025). "To CoT or not to CoT?" — meta-analysis of 100+ papers; chain-of-thought helps mainly on math and symbolic tasks. ICLR 2025. arXiv:2409.12183.
- Yang et al. (2025; revised 2026). Requirements-aware prompt optimization: models infer unspecified requirements correctly only 41.1% of the time; requirements-aware optimization +4.8% on average. arXiv:2505.13360.
- Kalai, A. T., et al. (OpenAI, 2025). "Why Language Models Hallucinate." arXiv:2509.04664.

- Cheng, et al. (2026). Sycophancy across 11 models (+49%). Science. — with the follow-up “social sycophancy” work from the same lab.
- Chain-of-Verification (CoVe). ACL Findings 2024. — self-check against named criteria, blinded from generation (FactScore 55.9→71.4).
- OPRO (ICLR 2024) and GEPA (ICLR 2026, arXiv:2507.19457). — metaprompting: letting the model draft and improve the prompt.
- OpenAI Instruction Hierarchy (2024). arXiv:2404.13208. — the chain of command behind system/developer/user layers.

Vendor guidance (fetched August 2026 unless dated)

- **Anthropic** — Prompting best practices; long-context tips; reduce-hallucinations guide; “Building Effective Agents”; context-engineering guidance; Claude Code documentation.
- **OpenAI** — GPT-5 prompting guide; reasoning best practices; “A Practical Guide to Building Agents”; the Model Spec (model-spec.openai.com, 2025-09-12 edition, checked 2026-08-18).
- **Google** — Gemini for Workspace prompting guides (Persona · Task · Context · Format).
- **Microsoft** — Copilot prompting guidance (Goal · Context · Source · Expectations); Copilot agents documentation; Copilot-in-Outlook behavior verified September 2026 (research note r28).

Standards, governance, and security

- NASA — “Appendix C: How to Write a Good Requirement” (nasa.gov/reference/appendix-c-how-to-write-a-good-requirement/).
- INCOSE — Guide to Writing Requirements, version 4 summary sheet (incose.org).
- OWASP — LLM Top 10 (2025) and “Top 10 for Agentic Applications for 2026” (genai.owasp.org, fetched 2026-09-10).
- UK NCSC — “Thinking carefully before adopting agentic AI” (ncsc.gov.uk) and “Prompt injection is not SQL injection (it may be worse)” (Dec 8, 2025).
- Singapore IMDA — responsible-deployment case study; CSA advisory AD-2026-005 (2026-05-28).

Landscape and benchmarks (dated snapshots — refresh quarterly)

- Stanford HAI — AI Index 2026 (adoption and capability-gap figures, cited as of Mar/Apr 2026).
- Artificial Analysis — model leaderboard (artificialanalysis.ai/leaderboards/models), transcribed 2026-09-13 for the speed-vs-capability chart; three independent fetches agreed.
- LMArena and SWE-bench Verified — August–September 2026 snapshots (vendor launch figures named as such).
- Product dates and pricing — vendor launch posts and business reporting (Reuters, CNBC, TechCrunch, Bloomberg), each dated where cited.

Books (the recommended-reading shelf)

- Mollick, E. — **Co-Intelligence** (2024). The working-with-AI mindset book; read first.
- Kneusel, R. — **How AI Works** (2024). The concepts of Part 1, gently and rigorously.
- Phoenix, J., and Taylor, M. — **Prompt Engineering for Generative AI** (O’Reilly, 2024). The five principles applied; closest to Parts 3–4.
- Berryman, J., and Ziegler, A. — **Prompt Engineering for LLMs** (O’Reilly, 2024). How prompts actually meet the model.

- Huyen, C. — **AI Engineering: Building Applications with Foundation Models** (O'Reilly, 2025). RAG, agents, evaluation — the engineering behind Part 5.
- Alammari, J., and Grootendorst, M. — **Hands-On Large Language Models** (2024). Visual internals: tokens, embeddings, transformers.

KEY — The fastest teacher of all

A real task. Adapt a course example, run it, check the result against its source, and save the improved prompt. The bibliography is where the evidence lives; the habit is what the course is for.

The package

Everything the course produces lives in two folders. The deliverables/ folder holds the five artifacts you will open and reuse:

- **From_Prompts_to_Agents_Facilitated_60_Minute.pptx** — the facilitated deck, with full speaker notes.
- **Course_Workbook.xlsx** — the exercises: staged prompts, self-checks, the supplier dataset, and the PLAYBOOK tab.
- **Prompt_Template_Creator.html** — the interactive builder on the element taxonomy: pick an artifact, choose requirement types, assemble the prompt.
- **From_Prompts_to_Agents_Course.pdf** — this book: the course, readable with no session attended.
- **From_Prompts_to_Agents_Cheat_Sheet.pdf** — the one-page distillation for the desk.

Everything else sits under references/: Elements_of_Prompting_Field_Guide.pdf with the full evidence compendium, Prompt_Element_Taxonomy_Reference.pdf, and Prompt_Template_Configurator.xlsx. The exercise sources are in exercise-data/: the supplier materials (the clean CSV, the dashboard, the mock presentation), the email thread, the three quotation PDFs, the plain-language sources, and the research pack. The build sources sit in src/ beside the package root. The supplier scenario (Alpha Components, Bravo Plastics, Cardinal Metals) is entirely fictional.

CHECK — Leaving the appendix

You are done with this course when you can name the requirement types your next artifact needs, write them into a brief, and point at the check that would catch the most likely failure.

APPENDIX A · PRACTICE MATERIAL

The practice pack: every exercise source, in full

Everything the exercises consume, printed verbatim so this book is self-contained: the supplier dataset, the email thread, the three quotations, the research pack, and the plain-language sources with their samples. The same content ships as separate files in `references/exercise-data/` — use the files when working, this appendix when reading. All names, companies, and numbers are fictional.

A.1 · The supplier dataset (Supplier_Data_Exercise.xlsx)

The single input for the data arc, printed in full: 144 detail rows plus the deliberate TOTAL row, one row per month × site × supplier, period 2025-09 to 2026-08. The two planted quirks are here to find: the TOTAL row (exclude it from every calculation, reconcile against it), and one blank Inspection_Hours cell (missing, not zero). All data fictional and seeded — stable across rebuilds and identical to the shipped file.

Month	Site	Supplier	Shipped	Ret.	Def.	Hrs	Cost USD	OT %
2025-09	Austin	Alpha Components	1781	2	8	22.8	12.67	89.2
2025-09	Austin	Bravo Plastics	1590	8	14	15	9.85	86.3
2025-09	Austin	Cardinal Metals	1569	1	13	34.8	13.89	87.4
2025-09	Berlin	Alpha Components	1291	0	9	36.3	12.16	92.1
2025-09	Berlin	Bravo Plastics	1564	6	14	17	10.09	91.1
2025-09	Berlin	Cardinal Metals	1499	1	13	45.1	14.22	91.8
2025-09	Osaka	Alpha Components	2167	1	14	41.6	12.64	98.4
2025-09	Osaka	Bravo Plastics	1947	5	16	21.4	9.7	97.4
2025-09	Osaka	Cardinal Metals	2088	3	13	25.8	13.94	99.7
2025-09	Monterrey	Alpha Components	1226	0	9	37.3	12.36	85.8
2025-09	Monterrey	Bravo Plastics	1317	3	22	31.8	10.08	88.5
2025-09	Monterrey	Cardinal Metals	1087	2	8	22.6	14.61	85.2
2025-10	Austin	Alpha Components	1863	1	12	35	12.82	89.7
2025-10	Austin	Bravo Plastics	1645	8	20	22.4	10.12	92.6
2025-10	Austin	Cardinal Metals	1698	1	14	45.9	13.77	93.4

Month	Site	Supplier	Shipped	Ret.	Def.	Hrs	Cost USD	OT %
2025-10	Berlin	Alpha Components	1469	1	9	27.5	12.36	94.3
2025-10	Berlin	Bravo Plastics	1575	2	23	45.1	9.51	93.4
2025-10	Berlin	Cardinal Metals	1443	0	11	34.5	14.11	91
2025-10	Osaka	Alpha Components	2014	1	14	42	12.46	98.9
2025-10	Osaka	Bravo Plastics	2184	2	26	34.4	9.61	100
2025-10	Osaka	Cardinal Metals	2140	1	16	38.9	13.68	96.8
2025-10	Monterrey	Alpha Components	1302	0	12	40.6	12.08	88.7
2025-10	Monterrey	Bravo Plastics	1290	10	17	16.7	9.55	88.5
2025-10	Monterrey	Cardinal Metals	1110	1	11	31.6	14.54	81.2
2025-11	Austin	Alpha Components	1706	1	10	31.3	12.2	88.3
2025-11	Austin	Bravo Plastics	1785	3	26	30.6	9.66	100
2025-11	Austin	Cardinal Metals	1677	4	12	21.4	13.75	90.5
2025-11	Berlin	Alpha Components	1549	4	8	16.3	12.61	92.9
2025-11	Berlin	Bravo Plastics	1351	1	18	43.9	9.83	94.8
2025-11	Berlin	Cardinal Metals	1344	2	9	27.8	14.14	90.5
2025-11	Osaka	Alpha Components	2126	1	13	45.5	12.38	94.9
2025-11	Osaka	Bravo Plastics	1931	4	21	28.4	9.7	100
2025-11	Osaka	Cardinal Metals	1931	3	15	28.2	14.22	100
2025-11	Monterrey	Alpha Components	1280	1	12	43.6	12.17	92.8
2025-11	Monterrey	Bravo Plastics	1196	3	17	26.8	9.77	91.1
2025-11	Monterrey	Cardinal Metals	1268	3	11	27.5	14.66	87.7
2025-12	Austin	Alpha Components	1314	3	7		12.87	84.7
2025-12	Austin	Bravo Plastics	1364	2	19	32.7	10.09	86.2
2025-12	Austin	Cardinal Metals	1210	4	8	16.6	14.26	87
2025-12	Berlin	Alpha Components	993	2	6	21.9	12.57	88.7
2025-12	Berlin	Bravo Plastics	1057	3	16	30.7	9.65	89.7
2025-12	Berlin	Cardinal Metals	1052	2	7	23.6	14.42	88.6

Month	Site	Supplier	Shipped	Ret.	Def.	Hrs	Cost USD	OT %
2025-12	Osaka	Alpha Components	1394	0	8	46	12.09	93.4
2025-12	Osaka	Bravo Plastics	1657	2	26	42.1	9.61	94.5
2025-12	Osaka	Cardinal Metals	1626	1	13	33.6	14.32	96.7
2025-12	Monterrey	Alpha Components	863	0	7	44	12.13	84.4
2025-12	Monterrey	Bravo Plastics	829	5	12	22.7	10	85.6
2025-12	Monterrey	Cardinal Metals	857	0	9	43	13.77	89.4
2026-01	Austin	Alpha Components	1706	5	7	14.1	12.58	91.6
2026-01	Austin	Bravo Plastics	1751	11	19	14.3	10.11	96.9
2026-01	Austin	Cardinal Metals	1873	1	17	40.7	13.82	93.8
2026-01	Berlin	Alpha Components	1547	1	11	41.5	12.21	95.2
2026-01	Berlin	Bravo Plastics	1361	2	20	39	9.78	93.2
2026-01	Berlin	Cardinal Metals	1556	1	15	45.2	14.55	93.2
2026-01	Osaka	Alpha Components	1840	3	10	24.6	12.72	99.4
2026-01	Osaka	Bravo Plastics	1938	14	22	17.5	10.09	94.2
2026-01	Osaka	Cardinal Metals	1920	2	14	28.7	14.46	97.1
2026-01	Monterrey	Alpha Components	1126	0	9	41.7	12.27	88.9
2026-01	Monterrey	Bravo Plastics	1233	2	24	45.4	10.15	95.9
2026-01	Monterrey	Cardinal Metals	1122	3	10	22.5	14.11	95
2026-02	Austin	Alpha Components	1812	1	12	33.4	12.36	90
2026-02	Austin	Bravo Plastics	1805	2	28	43.6	10	90.9
2026-02	Austin	Cardinal Metals	1817	3	14	25.7	14.33	90.7
2026-02	Berlin	Alpha Components	1533	2	9	23.6	12.38	95.5
2026-02	Berlin	Bravo Plastics	1405	8	16	18.1	10.15	92.1
2026-02	Berlin	Cardinal Metals	1466	1	11	31.5	13.96	90.1
2026-02	Osaka	Alpha Components	2004	1	12	35	12.41	97.5
2026-02	Osaka	Bravo Plastics	2196	11	27	19.4	9.86	99.2
2026-02	Osaka	Cardinal Metals	2087	1	15	40.2	14.34	97.2

Month	Site	Supplier	Shipped	Ret.	Def.	Hrs	Cost USD	OT %
2026-02	Monterrey	Alpha Components	1085	1	7	29.2	12.09	90.6
2026-02	Monterrey	Bravo Plastics	1179	9	19	20.2	9.85	88.2
2026-02	Monterrey	Cardinal Metals	1081	0	12	38.6	14.1	93.8
2026-03	Austin	Alpha Components	1620	1	12	30.6	12.24	93.9
2026-03	Austin	Bravo Plastics	1837	3	36	45.6	10.16	89.3
2026-03	Austin	Cardinal Metals	1869	1	17	37	14.5	85.9
2026-03	Berlin	Alpha Components	1503	2	8	22.3	12.58	92.7
2026-03	Berlin	Bravo Plastics	1509	2	27	34.1	9.56	93.6
2026-03	Berlin	Cardinal Metals	1381	4	8	18.4	14.36	95.6
2026-03	Osaka	Alpha Components	2117	2	10	27	12.39	97.6
2026-03	Osaka	Bravo Plastics	2197	3	36	36.3	10.03	99
2026-03	Osaka	Cardinal Metals	1984	3	11	25.3	14.52	99
2026-03	Monterrey	Alpha Components	1259	0	10	33.3	12.22	86.8
2026-03	Monterrey	Bravo Plastics	1184	6	18	24.8	9.78	90.3
2026-03	Monterrey	Cardinal Metals	1110	5	7	14.9	14.23	91.9
2026-04	Austin	Alpha Components	1569	4	8	18.3	12.07	90.5
2026-04	Austin	Bravo Plastics	1673	9	30	26.1	10.08	93.4
2026-04	Austin	Cardinal Metals	1647	5	8	14.6	14.66	89.4
2026-04	Berlin	Alpha Components	1513	1	11	38.1	12.2	96.3
2026-04	Berlin	Bravo Plastics	1295	8	15	18	9.89	94.6
2026-04	Berlin	Cardinal Metals	1509	1	11	33.4	13.79	97
2026-04	Osaka	Alpha Components	2163	4	10	17.5	12.3	94.8
2026-04	Osaka	Bravo Plastics	1991	15	24	16.4	9.94	92.5
2026-04	Osaka	Cardinal Metals	1871	1	15	41.2	13.8	100
2026-04	Monterrey	Alpha Components	1163	2	8	25.9	12.32	87.3
2026-04	Monterrey	Bravo Plastics	1283	14	22	17.4	9.94	91

Month	Site	Supplier	Shipped	Ret.	Def.	Hrs	Cost USD	OT %
2026-04	Monterrey	Cardinal Metals	1184	1	13	44.9	14.47	82.5
2026-05	Austin	Alpha Components	1746	2	11	27.9	12.26	95.4
2026-05	Austin	Bravo Plastics	1590	11	28	21.8	9.92	87.5
2026-05	Austin	Cardinal Metals	1691	1	13	41.2	13.89	92.8
2026-05	Berlin	Alpha Components	1542	2	9	22.8	12.85	92.3
2026-05	Berlin	Bravo Plastics	1314	5	25	29.6	9.98	92.6
2026-05	Berlin	Cardinal Metals	1517	4	8	21.1	13.98	92.2
2026-05	Osaka	Alpha Components	2066	1	12	31	12.68	95.5
2026-05	Osaka	Bravo Plastics	1940	10	22	17.9	9.59	99.7
2026-05	Osaka	Cardinal Metals	2047	5	12	19.9	14.15	96.1
2026-05	Monterrey	Alpha Components	1205	3	6	17.2	12.23	77.2
2026-05	Monterrey	Bravo Plastics	1123	2	21	31.1	10.17	82.2
2026-05	Monterrey	Cardinal Metals	1110	2	11	29.7	14.24	86
2026-06	Austin	Alpha Components	1722	3	9	19.9	12.84	86.5
2026-06	Austin	Bravo Plastics	1698	2	31	34.1	9.61	92.8
2026-06	Austin	Cardinal Metals	1755	1	15	45.8	14.43	87.8
2026-06	Berlin	Alpha Components	1354	1	8	28.4	12.77	91.8
2026-06	Berlin	Bravo Plastics	1573	2	29	33.9	10.01	95.3
2026-06	Berlin	Cardinal Metals	1350	3	8	19	13.75	93.6
2026-06	Osaka	Alpha Components	1831	1	11	33	12.23	96.2
2026-06	Osaka	Bravo Plastics	2014	3	38	43.6	9.93	94
2026-06	Osaka	Cardinal Metals	2098	3	16	27.6	14.32	100
2026-06	Monterrey	Alpha Components	1097	2	6	23.8	12.52	83.9
2026-06	Monterrey	Bravo Plastics	1149	2	25	36	10.15	83.3
2026-06	Monterrey	Cardinal Metals	1099	1	8	27.8	13.92	88
2026-07	Austin	Alpha Components	1591	1	13	40.8	12.85	96.5

Month	Site	Supplier	Shipped	Ret.	Def.	Hrs	Cost USD	OT %
2026-07	Austin	Bravo Plastics	1653	2	31	38.2	9.95	92
2026-07	Austin	Cardinal Metals	1877	1	15	32.7	14.16	94.3
2026-07	Berlin	Alpha Components	1385	0	10	41.4	12.63	95
2026-07	Berlin	Bravo Plastics	1372	2	32	40	9.82	93.3
2026-07	Berlin	Cardinal Metals	1568	2	12	30.5	14.33	95.6
2026-07	Osaka	Alpha Components	2050	4	9	20	12.68	94.6
2026-07	Osaka	Bravo Plastics	1912	3	36	43.7	9.98	94.6
2026-07	Osaka	Cardinal Metals	1879	1	16	33.5	13.83	94.6
2026-07	Monterrey	Alpha Components	1102	0	10	43.2	12.13	86.4
2026-07	Monterrey	Bravo Plastics	1279	2	32	36.9	9.69	93.7
2026-07	Monterrey	Cardinal Metals	1138	3	10	26.3	14.18	78.3
2026-08	Austin	Alpha Components	1864	4	12	22.8	12.39	95.3
2026-08	Austin	Bravo Plastics	1844	11	31	23.6	10.19	90.6
2026-08	Austin	Cardinal Metals	1729	1	13	31.2	14.22	88.8
2026-08	Berlin	Alpha Components	1382	0	8	36.3	12.23	93.4
2026-08	Berlin	Bravo Plastics	1512	2	28	37.8	10.06	91.8
2026-08	Berlin	Cardinal Metals	1525	2	14	29.9	14.17	95.9
2026-08	Osaka	Alpha Components	2070	1	14	42	12.48	93.9
2026-08	Osaka	Bravo Plastics	1996	14	29	18.9	10.17	97.9
2026-08	Osaka	Cardinal Metals	1967	1	16	34.8	14.52	98.2
2026-08	Monterrey	Alpha Components	1155	1	10	38.3	12.06	85
2026-08	Monterrey	Bravo Plastics	1096	3	29	43.5	10.02	86.3
2026-08	Monterrey	Cardinal Metals	1232	3	12	28.2	14.57	88.7
TOTAL			224902	432	2207			

Columns as in the file: Month, Site, Supplier, Units_Shipped, Units_Returned, Defects_Found, Inspection_Hours, Unit_Cost_USD, On_Time_Percent.

A.2 · The email thread (Packaging_Change_Thread.txt)

The fictional ten-message conversation for the email exercise, verbatim — paste it into any approved assistant with the structured brief from chapter 4, step 6.

PACKAGING-CHANGE CONVERSATION – fictional training thread (copy-paste source)

Ten messages, oldest first. The attachment mentioned in message 5 is intentionally NOT supplied. No real people, companies, or projects. Matches the reference pages of the facilitated course deck.

1. Sep 7, 09:00 – Maya, project lead

Target the packaging change for September 25. Luis, please confirm the trial plan.

2. Sep 7, 10:20 – Luis, operations

I can run the trial on September 18, subject to quality review.

3. Sep 8, 08:40 – Priya, finance

Approved provided the total trial cost does not exceed 18,000 and quality signs off.

4. Sep 8, 11:00 – Ben, quality

The revised drawing is mentioned in the plan. Please send it before I can approve the trial.

5. Sep 9, 13:15 – Elena, supplier contact

The updated drawing is attached. The material shipment will arrive on September 23.

[Attachment referenced: revised packaging drawing – NOT supplied with this exercise]

6. Sep 9, 15:00 – Maya, project lead

September 25 is no longer feasible. The proposed change date is now October 2. Luis, update the trial plan by September 16.

7. Sep 10, 09:30 – Omar, logistics

Who will pay the extra freight for the later material shipment?

8. Sep 10, 14:10 – Luis, operations

I will submit the revised trial plan by September 16. The new trial date is September 25.

9. Sep 11, 08:50 – Priya, finance

The 18,000 cap still applies. I have not approved additional freight charges.

10. Sep 11, 11:20 – Maya, project lead

Plan for October 2, pending Ben's quality sign-off. Luis owns the revised trial plan. We still need the freight decision.

Verbatim copy of Packaging_Change_Thread.txt (references/exercise-data/).

A.3 · The supplier quotations (Quote_*.pdf)

The three quotation documents for steps 7–8, transcribed from the shipped PDFs (Quote_Alpha_Components.pdf, Quote_Bravo_Plastics.pdf, Quote_Cardinal_Metals.pdf). They are engineered to be non-comparable as quoted: different currencies, delivery terms, and inclusions.

Alpha Components Inc.
2200 Commerce Park Drive · Austin, TX · USA

QUOTATION – ref. AC-26-0912 · in response to RFQ-2026-114

Item: EN-450 anodized aluminum enclosure, per drawing rev. D. Requested quantity: 5,000 units
(option:
10,000).

Unit price USD 13.90 / unit at 5,000 pcs · USD 12.60 / unit at 10,000 pcs

Tooling USD 8,500 one-time (3 weeks)

Minimum order 1,000 units

Lead time 4 weeks after tooling approval

Payment terms Net 30

Warranty 12 months, workmanship

Shipping FOB Austin (buyer arranges freight)

Validity 60 days

Notes: Prices include standard black anodizing. Expedite program available (+8%, lead time 2 weeks).

Fictional document generated for the "From Prompts to Agents" training. Not a real company, offer, or price.

Transcription of Quote_Alpha_Components.pdf.

Bravo Plastics GmbH
Industriestraße 14 · Hamburg · Germany

QUOTATION – ref. BP-2026-3341 · in response to RFQ-2026-114

Item: EN-450 anodized aluminum enclosure, per drawing rev. D. Requested quantity: 5,000 units (option: 10,000).

Price EUR 11,900 per 1,000 units (5,000-unit tier) · EUR 10,950 per 1,000 units (10,000-unit tier)

Tooling EUR 14,000 – not included in the prices above

Minimum order 5,000 units

Lead time 9 weeks including tooling

Payment terms Net 60

Warranty 6 months

Shipping EXW Hamburg (buyer collects; freight and import duties not included)

Validity 30 days

Notes: Quoted in euros. Surface per DIN standard; color matching on request. Prices exclude VAT.

Fictional document generated for the "From Prompts to Agents" training. Not a real company, offer, or price.

Transcription of Quote_Bravo_Plastics.pdf.

Cardinal Metals Ltd.
Attercliffe Works · Sheffield · United Kingdom

QUOTATION – ref. CM/26/RFQ114 · in response to RFQ-2026-114
Item: EN-450 anodized aluminum enclosure, per drawing rev. D. Requested quantity: 5,000 units
(option:
10,000).

Unit price USD 16.40 / unit at 5,000 pcs · USD 15.20 / unit at 10,000 pcs

Tooling Waived for orders of 5,000 units or more

Minimum order 2,500 units

Lead time 6 weeks, all-in

Payment terms Net 45

Warranty 24 months, including anodizing defects

Shipping DDP your dock – freight and duties included

Validity 90 days

Notes: Price includes lot-level material certs and PPAP level 3 documentation.

Fictional document generated for the "From Prompts to Agents" training. Not a real company, offer,
or price.

Transcription of Quote_Cardinal_Metals.pdf.

A.4 · The research pack (research-pack/)

The four reference files for step 9, verbatim. Two of them disagree on review cadence and one carries no date — deliberately: the exercise is scored on keeping that visible.

Source 1 – Supplier Quality Manual, section 7 (excerpt)
Fictional Manufacturing Co. · Rev C · dated 2026-02-10 · pages 12-13

7.1 Each approved supplier shall be reviewed **quarterly**. The review shall cover: delivered quality (return rate computed as total units returned divided by total units shipped in the period), inspection findings (defects found at incoming inspection, reported separately from returns), on-time performance, and the status of open corrective actions.

7.2 Return rate and defect rate shall never be added together: they measure different failure points (escaped versus caught).

7.3 A supplier whose return rate exceeds twice the site average for two consecutive reviews shall be placed on a corrective-action plan with a named owner and a re-measurement date. The plan format is defined in section 9.

7.4 Data used in the review must reconcile to the receiving system's period totals before the meeting. Unreconciled figures shall not be presented.

Verbatim copy of src1_quality_manual_excerpt.md.

Source 2 – Plant quality newsletter (excerpt)
Fictional Manufacturing Co. · issue 2026-06 · page 2

"Since January the supplier board reviews scores **monthly** – the quarterly cycle hid slow drifts for up to ninety days," writes the plant quality lead. The newsletter credits the shorter cycle with catching one supplier's packaging issue two months earlier than the previous cadence would have.

It also reminds readers that a review pack "is a decision document, not a data dump": one page per supplier, the trend chart first, and a recommendation the plant manager can approve or reject in the meeting.

The newsletter does not say whether the monthly cadence applies to every supplier or only to those already on a corrective-action plan.

Verbatim copy of src2_quality_newsletter.md.

Source 3 - Supplier scorecard guideline (one-page internal aid)
Fictional Manufacturing Co. - undated

- Score each supplier on four measures: return rate, defect rate, on-time percent, and responsiveness to corrective actions.
- Weight by units shipped when combining periods; never average monthly percentages.
- Show numerator and denominator next to every rate.
- Missing data is shown as missing, never as zero.
- The scorecard states the reporting period and the data source on its face.
- Responsiveness is scored from corrective-action closure dates; the guideline does not define the scale.

Verbatim copy of src3_scorecard_guideline.txt.

Source 4 – Eight-Discipline (8D) problem solving, one-pager (fictional)
Fictional Manufacturing Co. · training aid · dated 2025-11-03 · page 1

When a supplier issue is confirmed, open an 8D: D1 team · D2 problem description with the data behind it (period, numerator, denominator) · D3 containment · D4 root cause · D5-D6 corrective action chosen and implemented · D7 prevention · D8 closure with re-measurement. A supplier review that raises an issue without opening a numbered 8D, or closes one without re-measurement, is incomplete.

The one-pager does not state who may close an 8D, and gives no target closure time.

Transcription of src4_8d_onepager.pdf.

A.5 · The plain-language pack (plain-language/)

The reference exercise's four topics — each file holds a SOURCE text to hand the assistant and a sample output produced with the plain-language prompt (chapter 4). Printed verbatim; run the prompt on a SOURCE before reading its sample.

inventory_replenishment.md

Topic: inventory replenishment (live demonstration)

SOURCE (give this to the assistant)

Inventory replenishment is the process of reordering stock before it runs out. A reorder point is the inventory level that triggers a new order; it is set by multiplying expected daily demand by the supplier lead time in days, then adding safety stock. Safety stock is extra inventory held to absorb demand spikes and late deliveries; more variability in demand or lead time requires more safety stock. Ordering too late causes stockouts and lost sales; ordering too early or too much increases holding costs (storage, capital tied up, spoilage or obsolescence risk). The reorder quantity is a separate decision from the reorder point and is often set to balance ordering costs against holding costs. These formulas assume demand history predicts future demand; a promotion, a new customer, or a supply disruption can invalidate that assumption, so reorder parameters should be reviewed periodically rather than set once.

SAMPLE OUTPUT (what a good result looks like)

The main idea

Reordering works best when you order **before** you run out – at a planned stock level called the reorder point – not when the shelf is already empty.

When to reorder

The reorder point is the stock level that tells you "order now." It has three parts: how much you use each day, how many days a new order takes to arrive, and a safety cushion on top. Example: if a bakery uses 10 bags of flour a day and delivery takes 4 days, it needs to reorder no later than 40 bags – plus that cushion, which the next section explains.

The extra cushion

That extra is called **safety stock**: spare inventory that protects you when you suddenly need more than usual, or when a delivery is late. The less predictable your needs or your supplier, the bigger the cushion should be.

The trade-off

Ordering too late means running out and losing sales. Ordering too much means paying to store things – and some stock can spoil or go out of date. How MUCH to order each time is a separate decision from WHEN to order.

An analogy: the reorder point is like your car's fuel light. It comes on early enough to reach a gas station. But the analogy stops there – a fuel light is set once by the carmaker, while a reorder point must be **reviewed regularly**, because it relies on the past predicting the future. A promotion, a new customer, or a supply problem can make yesterday's numbers wrong.

Check yourself

1. What sets the reorder point? *(Daily use multiplied by delivery days, plus safety stock – all three parts, per the source formula.)*
2. What is safety stock for? *(Demand spikes and late deliveries.)*
3. Why review reorder points regularly? *(They assume the past predicts the future; promotions, new customers, or disruptions break that assumption.)*

Fidelity note (part of the check)

Preserved: the point/quantity distinction, the variability rule, both failure costs, and the review caveat. No fixed numbers were invented; the bakery example illustrates the formula in the source.

Verbatim copy of plain-language/inventory_replenishment.md.

household_budgeting.md

Topic: household budgeting (self-study variation)

SOURCE (give this to the assistant)

A household budget is a plan that allocates expected income to expected expenses over a period, usually a month. Expenses are commonly split into fixed costs, which stay the same each period (rent, insurance, loan payments), and variable costs, which change with usage or choices (groceries, fuel, entertainment). A budget is balanced when planned expenses plus planned savings do not exceed expected income. Savings set aside for irregular but predictable expenses – annual insurance premiums, car maintenance, holiday gifts – are called sinking funds; without them, these expenses appear as surprises even though they are foreseeable. An emergency fund is different: it covers genuinely unpredictable events such as job loss or urgent repairs, and a common guideline is three to six months of essential expenses. Guidelines such as allocating 50% of income to needs, 30% to wants, and 20% to savings are starting points, not rules; households with high fixed costs or irregular income must adapt the ratios. A budget based on estimated numbers drifts from reality, so spending should be reviewed against the plan regularly, and the plan revised when income or circumstances change.

SAMPLE OUTPUT (what a good result looks like)

The main idea

A budget is a plan made **before** the money is spent: you decide where your income should go each month, instead of finding out afterwards where it went.

Two kinds of spending

Some costs are the same every month – rent, insurance, loan payments. These are called **fixed costs**. Other costs change depending on what you do – food, fuel, fun. These are **variable costs**. A budget lists both and adds planned savings. The plan works when all of it together is not more than the money coming in.

Saving for things you can see coming

Some expenses do not happen every month but are not surprises either: a yearly insurance bill, car repairs, holiday gifts. Money set aside a little at a time for these is called a **sinking fund**. Without one, a bill you could have predicted feels like an emergency.

Saving for things you cannot see coming

A real **emergency fund** is for events you cannot predict – losing a job, an urgent repair. A common guideline is enough to cover three to six months of essential costs. Guideline, not rule: the same is true of the well-known 50/30/20 split (needs/wants/savings). If your rent is high or your income changes month to month, your numbers will look different – and that is fine.

An analogy: a budget is like a shopping list. You write it before entering the store, so the store does not decide for you. But the analogy stops there – a shopping list is finished after one trip, while a budget must be **checked and revised regularly**, because it is built on estimates that drift away from real life.

Check yourself

1. What is the difference between fixed and variable costs? *(Fixed stay the same each period, like rent; variable change with usage or choices, like groceries.)*
2. What is a sinking fund for? *(Irregular but predictable expenses – foreseeable bills that would otherwise feel like surprises.)*
3. Is the 50/30/20 split a rule? *(No – a starting point; households with high fixed costs or irregular income must adapt the ratios.)*

Fidelity note (part of the check)

Preserved: the fixed/variable distinction, the sinking-fund vs emergency-fund distinction, the "guideline not rule" caveat on both the three-to-six-months figure and the 50/30/20 split, and the review caveat. No numbers were invented beyond those in the source.

Verbatim copy of plain-language/household_budgeting.md.

water_cycle.md

Topic: the water cycle (self-study variation)

SOURCE (give this to the assistant)

The water cycle describes the continuous movement of water on, above, and below Earth's surface. Solar energy drives evaporation from oceans, lakes, and soil; plants release additional water vapor through transpiration. As moist air rises and cools, vapor condenses into droplets, forming clouds. When droplets grow heavy enough they fall as precipitation – rain or snow depending on temperature. Precipitation either runs off into rivers and eventually the ocean, soaks into the ground to recharge groundwater, or is stored temporarily as ice and snow. Residence times vary enormously: water may stay in the atmosphere about nine days on average, but in glaciers or deep aquifers for centuries or longer. The cycle conserves total water: it changes form and location but is neither created nor destroyed in the process.

SAMPLE OUTPUT (what a good result looks like)

The main idea

Earth keeps reusing the same water, moving it in a loop between the sea, the sky, and the land.

How the loop works

The sun warms water in oceans and lakes and turns some of it into **water vapor** – water in the form of a gas. Plants add vapor to the air too. High in the sky, the vapor cools and turns back into tiny droplets that make clouds. When droplets get heavy, they fall as rain – or snow, when it is cold. That water then flows to rivers and the sea, soaks into the ground, or is stored as ice, and the loop begins again.

The part people miss

Water does not move through the loop at one speed. A drop may spend about nine days in the air – but centuries locked in a glacier or deep underground.

An analogy: the cycle is like a giant recycling system – nothing is thrown away, everything is reused. The analogy stops here: real recycling changes products into new products, while the water cycle keeps the water itself the same and only changes its **form** (liquid, vapor, ice) and its **place**.

Check yourself

1. What powers the water cycle? **(The sun's energy.)**
2. Name the three places rain can go after it falls. **(Rivers/sea, into the ground, stored as ice or snow.)**
3. Is new water created in the cycle? **(No – the same water changes form and place.)**

Fidelity note (part of the check)

Preserved: transpiration (as "plants add vapor"), the residence-time contrast, and conservation of water. Simplification flagged in review: "about nine days" is an average, not a rule – the word "may" keeps that honest.

Verbatim copy of plain-language/water_cycle.md.

internet_message.md

Topic: how an internet message travels (self-study variation)

SOURCE (give this to the assistant)

When a message is sent over the internet, it is first divided into packets – small blocks of data, each carrying the destination address and a sequence number. Routers pass each packet toward its destination hop by hop, choosing a next step from routing tables; packets from the same message may take different paths and arrive out of order. The receiving device uses the sequence numbers to reassemble the message and requests retransmission of any packet that was lost. This design, packet switching, lets many conversations share the same lines efficiently and routes around failures, but it does not guarantee a fixed path or arrival time; delays vary with congestion. Protocols such as TCP provide the checking and reordering that make delivery reliable on top of this best-effort network.

SAMPLE OUTPUT (what a good result looks like)

The main idea

The internet does not send your message in one piece. It cuts the message into small pieces, sends each piece separately, and puts them back together at the other end.

The journey

Each piece is called a **packet** – a small block of data with the delivery address and its place in line written on it. Machines called **routers** pass each packet along, one hop at a time, each choosing the next step. Pieces of the same message can travel different roads and arrive in the wrong order. The receiving device uses the numbers to put them back in order – and asks for a new copy of any piece that got lost.

Why do it this way

Sharing the lines packet by packet lets millions of conversations use the same wires, and traffic can steer around broken links. The cost: no piece is promised a fixed road or arrival time – busy moments mean delays. A set of rules called **TCP** does the checking and re-ordering that makes delivery dependable anyway.

An analogy: it is like mailing a long letter as many postcards, each numbered, possibly traveling by different routes. The analogy stops here: the post office never notices a lost postcard, but the internet's rules detect a missing packet and automatically ask the sender for it again.

Check yourself

1. What two things are written on every packet? **(The destination address and its sequence number.)**
2. Why can pieces arrive out of order? **(They may take different paths.)**
3. What happens when a piece is lost? **(The receiver requests it again.)**

Fidelity note (part of the check)

Preserved: best-effort delivery, variable delay under congestion, and TCP as the reliability layer. Nothing promises speed or a fixed route – the source's "no guarantee" condition survives in "no piece is promised a fixed road."

Verbatim copy of plain-language/internet_message.md.

APPENDIX B · ANSWER KEYS

Answer keys: check yourself after attempting

The expected results for every exercise, for self-study. Attempt each exercise first, then grade your result here. Reading a key before attempting removes exactly the practice the exercise exists to give.

B.1 · Supplier analysis — the worked key

The full expected results for the data arc, computed from the dataset (they rebuild with it).

Item	Expected result
Profile	144 detail rows; one TOTAL row to exclude (and reconcile against); one blank Inspection_Hours value (2025-12, Austin, Alpha Components) — missing, not zero.
Totals	224,902 units shipped · 432 returns (overall 0.192%) · 2,207 defect occurrences (a separate measure, never added to returns).
Ranking	Bravo Plastics 0.348% (262 / 75,184) — highest, ≈ 3.5× Alpha · Cardinal Metals 0.129% (96 / 74,658) · Alpha Components 0.099% (74 / 75,060).
Trend (follow-up)	Bravo is elevated and volatile, not steadily worsening: peak ≈0.737% (2026-04), quiet months ≈0.14% (2026-06/07), ≈0.465% (2026-08). Patterns only — any causal claim fails.
Denominator lesson	No overall on-time delivery rate can be computed: On_Time_Percent has no delivery counts behind it. An averaged percentage must be labeled an unweighted mean of monthly percentages.
Dashboard check	Berlin × Bravo Plastics, 2026-03..2026-08 → exactly 6 records, 8,575 units, 21 returns, 0.245% — in tiles, chart, AND table.
Wrong turns	Averaging monthly percentages · adding defects to returns · treating the blank as zero · including the TOTAL row · claiming a cause the data cannot show.

B.2 · Email brief — the expected result and the four traps

The expected brief for the packaging-change thread, and the four traps it must catch.

Expected current-state brief. The packaging change is now planned for October 2 (superseding September 25), pending Ben's quality sign-off, which itself waits on review of the revised drawing. The trial is targeted for September 25 (Luis, message 8 - superseding the original September 18), with Luis's revised trial plan due September 16 and the material shipment arriving September 23. Finance approval stands only within the 18,000 cost cap, and freight responsibility for the later shipment is unresolved.

Item	Status	Conditions	Evidence
Change date October 2	Proposed/planned - NOT finally approved	Ben's quality sign-off	msg 6, msg 10

Item	Status	Conditions	Evidence
Trial budget	Approved	$\leq 18,000$ AND quality sign-off; no freight extras approved	msg 3, msg 9
Trial date September 25	Current plan (supersedes Sep 18)	revised plan due Sep 16	msg 2 → msg 8
Original change date September 25	Superseded	-	msg 1 → msg 6

Decisions vs. proposals.

Task	Owner	Due	Status	Dependency	Evidence
Submit revised trial plan	Luis	Sep 16	Committed	-	msg 6, 8, 10
Review drawing, quality sign-off	Ben	Not stated	Open	access to the drawing	msg 4, 10
Decide freight responsibility	Not stated	Not stated	Open	-	msg 7, 9, 10
Provide revised drawing to Ben	Elena/Maya	Not stated	Claimed sent; attachment not accessible	-	msg 5

Expected action table.

- **Changed dates** - change date Sep 25 → Oct 2 (msg 6); trial date Sep 18 → Sep 25 (msg 8). Oct 2 is the change date, Sep 25 the trial date - do not conflate.
- **Conditional approval** - Priya's approval is conditional ($\leq 18,000$ + quality sign-off) and explicitly excludes extra freight (msg 9). "Approved" alone is wrong.
- **Unresolved question** - freight responsibility (msg 7) is never answered.
- **Missing attachment** - the revised drawing is referenced (msg 5) but not supplied; its content must not be summarized, and Ben's sign-off still depends on it.
- Also correct: "Not stated" for the freight owner and Ben's due date; no final approval of October 2 exists anywhere in the thread; any reply is a draft for human review, never sent automatically.

B.3 · Quotation comparison — the key

The expected extraction and normalized values — and the expected outcome: no best quote while the basis is incomplete.

SELF-CHECK – quotation extraction and comparison

Step 1 (extraction) is complete when every commercial value is verbatim, cited to its source file, and gaps read "Not stated" – notably: Alpha freight amount and taxes; Bravo freight, duties, and the EUR→USD basis; Cardinal taxes.

Step 2 (comparison) at the 5,000-unit tier, tooling amortized over 5,000:

- Alpha: $\text{USD } 13.90 + 8,500/5,000 = \text{**USD } 15.60 \text{ / unit**}$, freight NOT included.
- Bravo: $\text{EUR } 11,900/1,000 + 14,000/5,000 = \text{**EUR } 14.70 \text{ / unit**}$, freight and duties NOT included, VAT excluded – NOT convertible without a supplied rate.
- Cardinal: $\text{USD } 16.40 + 0 = \text{**USD } 16.40 \text{ / unit**}$, freight and duties INCLUDED (DDP), certs + PPAP included, longest warranty and validity.

The correct conclusion is that NO best quote can be named yet: the blocking gaps are the EUR→USD basis, Alpha/Bravo freight (and Bravo duties), and tax treatment. A submission that crowns a winner without resolving these fails the exercise; one that ranks by stated unit price alone falls for the trap.

Expected extraction gaps, normalized values, and the expected outcome.

B.4 · Research workbook — the key

What a complete evidence workbook shows, including the planted conflict kept visible.

SELF-CHECK – research workbook

Traceability: every Evidence row cites file + page/section and quotes a short excerpt; nothing appears that cannot be traced (12 expected rows, ±2).

Must-catch items:

1. The CADENCE CONFLICT – manual quarterly (2026-02) vs newsletter monthly (2026-06). Kept as two rows, surfaced in Synthesis as a decision, never silently merged.
2. "Not stated" entries – undated guideline; responsiveness scale; 8D closure authority and target time; monthly-cadence scope; §9 plan format.
3. Separate-measures and sums-not-averages rules carried into the synthesis.
4. Synthesis ordered foundations → content → escalation → conflict → gaps → implication; Sources sheet indexes all four files.

A workbook that invents a fifth source, fills gaps with guesses, or resolves the cadence conflict by picking one side without flagging it fails the check.

What a complete evidence workbook must show.

APPENDIX C · THE CHEAT SHEET

The one-page cheat sheet

The next page is the printable course cheat sheet — the requirement skeleton, the requirement-type menu, the quality bar, a before/after example, acceptance checks, ASK vs DELEGATE, and the works/myth/expired box, on one page. It is identical to the separate deliverables/From_Prompts_to_Agents_Cheat_Sheet.pdf; print that file at full size for desks.

A prompt is a small work specification

WRITE IT — like a requirement

task and audience first · then only the requirements that resolve ambiguity

THE SKELETON — EVERY PROMPT

Purpose & audience: help [reader] make [decision] or complete [task].

Sources & scope: use [source / version / period]; include [scope]; exclude [items].

Selected requirements: only the types that resolve ambiguity for THIS artifact — a simple task needs a few, not all.

Acceptance evidence: check [specific result] against [source, calculation, or observable test].

Missing info & finish line: if [gap]: write “Not stated” and list it; if [conflict]: ask or stop. Answer, report limitations, then stop. Actions needing review: [actions].

REQUIREMENT TYPES — THE MENU (pick, don’t fill)

Functional behavior — what it must do: filter, calculate, extract, compare, explain.

Data & provenance — allowed inputs, definitions, period, traceability.

Scope & exclusions — in, out, and explicitly outside the task.

Audience & use — who uses it, for which decision.

Role & working behavior — a perspective plus observable conduct; a title alone is weak.

Content & completeness — required topics, fields, limitations.

Method & business rules — formulas, denominators, missing-data treatment.

Structure & interface — sections, tabs, slides, columns, controls.

Tone & presentation — voice, reading level, units, formats.

Accessibility & usability — labels, contrast, keyboard, understandable language.

Quality & acceptance — observable correctness and completeness criteria.

Constraints & authority — allowed tools, approvals, stop conditions.

Delivery & compatibility — file type, editability, offline use, destination.

Maintenance & reuse — refresh procedure, versions, reusable parameters.

REQUIREMENT QUALITY — IS EACH ONE...

necessary · clear · complete enough · consistent · feasible · singular · **verifiable**. A requirement you cannot check is a wish. Separate required outcomes from preferences; make trade-offs explicit.

BEFORE / AFTER — THE SUPPLIER CASE

Before: “Analyze the supplier data and tell me which supplier is worst.” — the model picks the rows, the metric, and “worst” for you, silently.

After — inspect: “Before calculating anything, inspect the file. Tell me what one row represents, how many detail rows it contains, whether any values are missing, and whether any total or summary row could be counted twice. Tell me what you would include or exclude, then wait.”

After — analyze: “Using detail rows only, calculate each supplier’s return rate as total Units_Returned divided by total Units_Shipped. Rank suppliers from highest to lowest. Show the totals used, reconcile the overall totals to the source file, and state one limitation of the comparison.”

Why it wins: each added line is a requirement type fixing one named weakness — and nothing is calculated before the scope is agreed.

RUN IT — the course tasks, one habit

produce the artifact, then check it against its own acceptance evidence

THE COURSE TASKS (deck order · workbook tab)

Analyze data — inspect first, then wait; return rate from totals, detail rows only; rank, reconcile to the source, state one limitation; follow-up = findings, not causes · 1-Analyze-Data.

Excel charts — the returned workbook: Summary sheet, native editable charts, traceable formulas, totals reconciled before it returns · Excel-Charts.

Build a dashboard — describe the behavior: date range, supplier and site filters, metric selector, trend chart, sortable table, reset; every filter updates every view · 2-Build-Dashboard.

Present findings — five slides, one main message each; findings separated from recommendations; no invented causes or commitments · 3-Present-Findings.

Summarize email — current status; decisions WITH conditions; owners and due dates; “Not stated” for gaps; cite sender and date; send nothing · 4-Summarize-Email.

Extract quotes — values verbatim, original currencies, “Not stated” when absent, source file and page per value; no ranking yet · Quote-Extract.

Compare quotes — normalize only with a supplied basis; formulas visible; no best quote until the basis is complete · Quote-Compare.

Self-study — research workbook (Research) · explain a topic clearly (5-Explain-Clearly) — same discipline; Reference Deck.

ACCEPTANCE CHECKS — BEFORE YOU TRUST IT

The loop — PLAN the task and the success check · DO run the prompt against the supplied source · CHECK the result against evidence or a known calculation · ACT: fix the specific failure and rerun the affected check.

Trace — one output number back to its source records.

Recompute — one calculation independently (code or by hand).

Reconcile — totals against a number you already trust.

Exercise — every requested behavior once (filter, sort, reset, empty case).

Coverage — compare against the requirement list; missing = not done.

Read as the reader — can the audience act on it? That is validation.

Never accept “looks done”: ask for evidence — the test it ran and what it returned.

ASK vs DELEGATE

ASK — request an answer or draft; you read, check, decide. The specification above is enough.

DELEGATE — assign bounded work: add the inputs register, the checks that stop the run, permissions, gates for irreversible steps, and the finished-report format.

The boundary: a prompt requests behavior; configured permissions and approvals restrict available actions. Reversible? proceed. Irreversible? ask.

SAVE WHAT WORKS

Each reusable prompt: name · owner · version · the example that proved it · its acceptance check.

Where: the Course Workbook carries every course prompt in a named tab, in deck order; the Prompt Template Creator (one HTML file) offers the same task families, plus explain-a-topic, with their requirement menus.

Refresh: facts about products and models expire — date them and re-verify before big reuse.